Generative AI (GenAI) Use Disclosure Statement at CICS

1. AI Tool(s) Used:
☑ ChatGPT ☑ Copilot ☐ Gemini ☐ Claude ☐ Grammarly
☐ Other/s: _____

2. Primary Purpose of Use: (Select all that apply)
☑ Ideation & Research (Brainstorming, topic exploration, literature search)
☑ Outlining & Drafting (Structuring ideas, initial text generation)
☑Editing & Refinement (Grammar correction, proofreading, style polish)
☑ Code & Logic Development (Code generation, debugging, refactoring)
☑ System & Data Analysis (Diagramming, system architecture, data processing)
☐ Image, audio, video, or other media generation
☐ Other/s: _____

3. Specific Section(s) & Scope of AI Use: (e.g., Section 2.1 Lit Review, Function X Debugging, Essay Outline)

a. Conceptualization of the topic (Problem Identification, Lit Review, Refinement of Project Objectives)

c. Manuscript Writing and Revision (Introduction, Methodology, Grammar Correction)

d. System Creation and Debugging (Code Generation and Explanation, Error Identification, Debugging)

e. Technical Guidance (Database Connection, System Deployment, Configuration, and Troubleshooting)

4. Verification Method(s) Applied: (Select all that apply)
☑ Citation & Source Check (Cross-checked sources in Google Scholar, IEEE, ACM, etc.)
☑ Content Verification (Read original source texts to confirm claims match retrieved data)
☑ Code Testing & Execution (Tested AI-generated code for syntax, logic, and vulnerabilities)
☑ Data & Math Review (Manually recalculated formulas, statistics, or transformations) ☑ Plagiarism Scan (Ran text through plagiarism software or unique phrase searches)
☐ Other: _____

5. Confirmation of Integrity
I confirm that I:
☑ Verified AI-generated information, references, code, and analysis before submission. ☑ Complied with copyright, attribution, and licensing requirements.
☑ Did not upload confidential University information, personal or research data, password, examination materials, or other restricted content into any AI tool.
☑ I accept full responsibility for my AI usage and recognize that the subject faculty holds discretion to evaluate, accept, or reject this submission based on its academic quality and integrity.

Full Name and Signature:

Dave Matthew O. Laman, Date: November 11, 2026



AI Use and Supporting Evidence

GitHub Copilot was used as an AI-assisted development tool during the development of the PigMENT web application. It helped generate initial frontend code, suggest interface improvements, and refine the layout and functionality of selected pages. The generated suggestions were reviewed and modified as needed.

The prompts below come from the supplied log. Existing tool attributions and validation notes are retained. Missing attributions and checks have not been inferred.

01 Initial Frontend Development

AI Tool: GitHub Copilot
Affected sections/files: index.html, my-pigs.html, and related CSS and JavaScript files.

Exact prompt

Act as a professional web developer helping me build a college-level web systems project called PigMENT (Pig Monitoring, Estimation, and Management Technology). PigMENT is a responsive web-based system intended for small-scale pig farmers in Marinduque. Its main purpose is to help farmers monitor pig growth, browse and post livestock listings, and access pig health information and geographic disease awareness reports.

For the initial prototype, use only:

- HTML5
- CSS3
- Vanilla JavaScript

Do not use React, Vue, or other frameworks. Do not implement the backend or database yet.

Create a clean, maintainable project structure with:

- index.html
- style.css
- script.js
- an assets folder for images and icons

Before writing the code, briefly explain the proposed structure and the purpose of each file. Then create the initial files.

Keep the code beginner-friendly, properly organized, and easy for a BSIT student to explain during a project defense.

Validation evidence:

- Opened the website locally to check whether the pages and interface loaded.
- Checked navigation between the available sections.
- Inspected the displayed sample pig records and dashboard information.● Reviewed the generated code and made adjustments where necessary.

02 Accessibility and Usability Improvements

AI Tool: GitHub Copilot

Affected sections/files: Dashboard, My Pigs page, digital pig-pen management interface, and related CSS files.

Exact prompt

Design the initial visual style of the PigMENT website.

The website should look like a practical agricultural technology platform—not a generic corporate dashboard and not an overly decorative AI-generated design.

Design direction:

- Professional, clean, trustworthy, and approachable
- Suitable for small-scale pig farmers and agricultural personnel
- Responsive on desktop, tablet, and mobile
- Use a restrained agriculture-inspired palette: deep green, warm off-white, light gray, and asubtle earthy accent
- Use readable typography, clear spacing, simple cards, and consistent buttons
- Avoid excessive gradients, glassmorphism, large decorative illustrations, and unnecessaryanimations
- Use semantic HTML and accessible color contrastCreate reusable CSS styles for:
- Colors and typography
- Page layout and containers
- Navigation
- Buttons
- Cards
- Form fields
- Status labels
- Tables
- Responsive behavior

Do not add functionality yet. Focus on establishing a consistent design system.

Validation evidence:

- Checked text readability and button visibility.
- Reviewed the layout of forms and available actions.
- Tested the interface at different screen sizes.
- Confirmed that primary controls use larger touch-friendly dimensions.

03 Online Pig Pen Management

Exact prompt

Act as a professional web developer. Extend PigMENT's existing Pig Growth Monitoring module by adding an Online Pig Pen Management feature.
The goal is to let farmers organize their pigs digitally according to the physical pens on their farm. This is especially useful when a mother pig gives birth and has several piglets that need to be managed together.

Main concept
Farmers should be able to create a digital pig pen, then assign one or more pigs to it. A pen may contain:

- A mother sow
- Her litter of piglets

03 Online Pig Pen Management (continued)

- Growing pigs
- Other pigs grouped together by the farmer

My Pigs page
Add a clear "Manage Pig Pens" action beside "Register a Pig."

Create a Pig Pens view that displays each pen as a large, easy-to-understand card. Each card should show:

- Pen name or number
- Pen type or purpose
- Number of pigs currently assigned
- Mother sow, if applicable
- Litter or group description
- Current status

Example fictional pens:

- Pen 01 — Sow and Newborn Litter — 1 sow, 8 piglets
- Pen 02 — Growing Pigs — 5 pigs
- Pen 03 — Empty Pen — Available

Include search or simple filtering only if it improves usability.

Create Pig Pen form
Create a simple form with:

- Pen name or number
- Pen type: Sow and Litter, Piglets, Growing Pigs, Fattening Pigs, or General
- Optional description
- Optional capacity
- Current status: Active or Empty

Keep the form beginner-friendly. Use clear labels and short helper text.

Assign pigs to a pen
Allow farmers to:

- Assign an existing pig to a pen
- Move a pig to another pen
- View the pigs assigned to a selected pen
- Remove a pig from a pen without deleting its pig record

When creating a sow-and-litter pen, allow the farmer to select the mother sow and add multiple piglets under the same litter.

Litter management
Add a way to record a litter with:

- Mother sow
- Birth date
- Number of piglets born
- Number currently alive● Optional notes

For the initial prototype, piglets may be recorded as individual pig records or as a litter group, but clearly indicate which approach is being used. Make it possible to assign the litter to its pen.

Important behavior

- Use fictional sample data for the prototype.● Use vanilla JavaScript only.

03 Online Pig Pen Management (continued)

- Keep data front-end only for now.
- Do not make buttons appear functional if they are not.
- Do not delete a pig record when it is removed from a pen.
- Make the interface usable for older and less tech-confident users.● Preserve the existing PigMENT design and project structure.

First explain the proposed user flow and data structure. Then implement the feature and list the files modified.

04 Dashboard and Pig Management Simplification

AI Tool: GitHub Copilot
Affected sections/files: index.html, my-pigs.html, and related CSS and JavaScript files.

Exact prompt

Create the initial PigMENT dashboard interface using the existing index.html, style.css, and script.js.

The dashboard should include:

1. A sidebar or responsive navigation with:
   ○ Dashboard
   ○ My Pigs
   ○ Marketplace
   ○ Health and Disease Awareness
   ○ Reports
   ○ Profile
2. A top section containing:
   ○ PigMENT logo or text-based wordmark
   ○ Page title

○ User profile area
○ Notification icon as a visual placeholder
3. Dashboard summary cards:
○ Total registered pigs
○ Pigs currently being monitored
○ Active marketplace listings
○ Recent health reports
4. A “Recent Activity” section with realistic but clearly fictional sample data.
5. A “Quick Actions” section with buttons for:
○ Register a pig
○ Create a marketplace listing
○ Submit a health report

Use sample data only. Clearly separate the interface from future backend functionality. Make the layout responsive and do not make the dashboard overcrowded.

05 Visual Design and Color Palette Refinement

AI Tool: GitHub Copilot Affected sections/files: Main stylesheet and shared interface components.

Exact prompt

Design the initial visual style of the PigMENT website. The website should look like a practical agricultural technology platform—not a generic corporate dashboard and not an overly decorative AI-generated design.

Design direction:

05 Visual Design and Color Palette Refinement (continued)

- Professional, clean, trustworthy, and approachable
- Suitable for small-scale pig farmers and agricultural personnel
- Responsive on desktop, tablet, and mobile
- Use a restrained agriculture-inspired palette: deep green, warm off-white, light gray, and asubtle earthy accent • Use readable typography, clear spacing, simple cards, and consistent buttons
- Avoid excessive gradients, glassmorphism, large decorative illustrations, and unnecessaryanimations

Use semantic HTML and accessible color contrast

- Create reusable CSS styles for:
- Colors and typography
- Page layout and containers
- Navigation
- Buttons
- Cards
- Form fields
- Status labels
- Tables
- Responsive behavior

Do not add functionality yet. Focus on establishing a consistent design system.

Validation evidence:

- Reviewed the updated colors and overall visual consistency.
- Checked the contrast between text, buttons, and backgrounds.
- Confirmed that the revised design remained readable across desktop and mobile layouts.

06 Marketplace Interface Development

AI Tool: GitHub Copilot
Affected sections/files: Marketplace page and its related CSS and JavaScript files. (Confirm the actual filenames used in the project.)

Exact prompt

Act as a senior UX/UI designer and front-end developer. Create the Marketplace page for PigMENT, a web based farm management system for small-scale pig farmers in Marinduque.

Before implementing, inspect the existing PigMENT files and follow the approved visual direction. The current design should be moving toward a richer, more polished interface with visual depth. Do not revert to the old pale green and cream palette, and do not make the marketplace look plain or generic.

The marketplace should help farmers browse livestock listings and create their own listings. Some users may be middle-aged or older and may have limited experience with technology, so the page must be visually appealing while remaining easy to understand.

1. Marketplace page layout

Create a clear page structure with:

- A visually strong page heading: “Pig Marketplace”
- A short explanation: “Browse pigs for sale from local farmers.”
- A prominent “Post a Listing” button
- A search bar with the placeholder: “Search by pig type, breed, or location”- Simple filters for:

- Pig type
- Breed
- Estimated weight
- Location
- A clearly visible way to reset filters

Do not display every filter as a separate large card. Keep the search and filtering area organized and easy to scan.

2. Livestock listing cards

Display fictional sample listings in an attractive, responsive grid.

Each listing card should include:

- A large, clear pig photo or image placeholder
- Listing title or pig type
- Breed
- Age

- Estimated weight, clearly labeled as an estimate
- Asking price - General location, such as barangay or municipality
- Seller name
- A clear “View Listing” action

Make the price and pig type easy to notice. Use visual hierarchy, spacing, image treatment, subtle shadows, borders, and contrasting surfaces to create depth.

Avoid overly small text, crowded card content, excessive icons, or too many decorative elements.

3. Listing details

When the user selects “View Listing,” show a simple listing details view or modal containing:

- Larger listing image
- Pig details
- Description
- Estimated weight
- Asking price
- General location
- Seller information
- A clear “Back to Listings” action

For the prototype, do not implement real messaging, payments, or transactions. If a contact action is shown, make it clearly a placeholder and do not pretend that a message was sent.

4. Create listing form

When the user selects “Post a Listing,” open a beginner-friendly form.

Include:

- Pig type or listing title
- Breed -

Sex

- Age
- Estimated weight
- Asking price
- Municipality or barangay

- Description
- Image upload area or image placeholder

Use simple labels and short helper text. Group related fields together and avoid displaying an intimidating wall of inputs.

Include clear “Cancel” and “Preview Listing” or “Save Listing” actions.

For the prototype:

- Use fictional sample listings
- Keep data front-end only
- Validate required fields
- Show clear success and error messages
- Do not claim the listing has been published online or saved to a database

5. Visual and accessibility requirements

Follow the approved PigMENT visual identity:

- Rich, distinctive agricultural technology aesthetic
- Deep forest green and rich teal as primary colors
- Warm amber or burnt orange for important highlights
- Neutral page background with contrasting content surfaces
- Strong visual hierarchy and noticeable depth
- Polished listing cards with purposeful use of shadows and bordersThe interface must:
- Use readable text and sufficient contrast
- Use large, clearly labeled buttons
- Avoid relying on icons alone
- Be usable on desktop, tablet, and mobile
- Keep the main user actions obvious
- Feel professional and welcoming—not childish, overly decorative, or like a generic template

6. Implementation instructions

- Use the existing HTML, CSS, and vanilla JavaScript files.
- Preserve the shared navigation and existing PigMENT layout.
- Do not introduce React, Vue, or another framework.
- Do not implement PHP or MySQL yet.
- Reuse existing styles and components where appropriate.
- Do not modify unrelated pages unless necessary for shared styling.
- Before coding, briefly explain the planned layout and interactions.
- After coding, list the files modified and provide a manual testing checklist.

Validation evidence:

- Tested marketplace search and filtering.
- Confirmed that selecting filters narrowed the displayed listings.
- Tested the reset filters function.
- Checked whether listing details could be opened and returned to the Marketplace.● Opened the listing form and checked its validation and preview.

07 Academic Writing and Research Section Refinement

AI Tool: ChatGPT Affected section/file: Chapter 1 – Section 1.4, Proposed Solution, Scope and Limitations.

Exact prompt

Act as an academic research writer assisting a second-year BSIT student in developing a research paper entitled:
'PigMENT: A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque.'

Write Section 1.4, Proposed Solution, Scope and Limitations, under Chapter 1: Introduction.

Use the following materials as your primary basis:

1. The provided initial project concept and system description.
2. The provided draft of Section 1.4.

3. The complete list of related literature and studies (RRL/RRS) with their corresponding IEEEreference numbers.

Writing Instructions

1. Preserve the original meaning, objectives, and proposed functions of PigMENT.
2. Improve the grammar, sentence construction, clarity, and academic tone of the provided draft.3. Use clear, formal, and straightforward academic English appropriate for a second-year BSIT research paper.
4. Avoid overly complicated vocabulary, unnecessary explanations, and exaggerated claims.
5. Organize the discussion into coherent paragraphs instead of using bullet points. 6. Maintain theoriginal section title and structure.

Required Content

A. Proposed Solution

- Introduce PigMENT as a web-based swine growth monitoring, marketplace, and diseaseawareness system.
- Identify small-scale and backyard pig farmers in Marinduque as the primary target users.● Explain how the system centralizes livestock records, growth monitoring, estimated selling values, marketplace access, and disease-related information.
- Briefly establish the relevance of web-based livestock management and record-keepingsystems through related studies.

B. Three Main System Features

1. Pig Growth Monitoring and Weight Estimation
   - Explain pig registration and recording of breed, age, sex, heart girth, and body length.
   - Discuss estimated live weight calculation and growth history.
   - Include estimated selling value based on the farmer's entered price per kilogram.
   - Support the feature using relevant studies on pig weight estimation.
2. Community Livestock Marketplace
   - Explain how farmers create livestock listings containing estimated weight, age, asking price,and general location.
   - Explain how potential buyers browse listings and contact sellers.
   - Support the feature using related studies on agricultural marketplaces and livestock supplychain platforms.
3. Pig Health Information and Geographic Disease Awareness
   - Explain the provision of basic pig health information, ASF symptoms, preventive measures, andbiosecurity practices.
   - Discuss the interactive map displaying published disease-related information in Marinduque.
   - Explain the relevance of disease awareness, biosecurity, and early reporting.● Include the local context of ASF monitoring in Marinduque.

C. Scope and Limitations

- Limit the project to a web-based system for small-scale and backyard pig farmers inMarinduque.
- Explain that farmers will manually enter livestock measurements and other information.
- Clarify that specialized sensors and automated weighing equipment are not included.● State that calculated live weights are estimates and depend on the estimation formula and accuracy of userprovided measurements.

- Clarify that the system will not diagnose diseases, predict outbreaks, or independently confirmdisease cases.
- Explain that farmer-submitted disease reports require administrative review and confirmationby authorized veterinary personnel.
- Explicitly exclude advanced features such as automated livestock monitoring, disease spreadprediction, and real-time diagnostic services.
- End by reinforcing the project's focus on its three primary features.

IEEE Citation Requirements
Use IEEE numbered in-text citations throughout the discussion.

Use the exact reference numbering provided in the supplied RRL list. Do not assign new numbers, invent references, or change the numbering.

Select only studies that directly support the claims being discussed.

Use appropriate citation groupings, such as:

- Pig weight estimation: [1], [2], [8]
- Marinduque ASF monitoring: [3]
- Agricultural marketplaces: [4], [13], [14]
- Wireless pig growth monitoring: [5]
- Biosecurity and disease reporting: [6], [15], [16]
- Web-based swine management: [10]
- Web-based pig record keeping: [20]

Do not force citations into statements they do not support. Place citations immediately after the claims they support.

Expected Output
Produce a complete, polished, and academically appropriate Section 1.4 titled: '1.4 Proposed Solution, Scope and Limitations' The result must:

- Consist of coherent academic paragraphs.
- Discuss the proposed system, its three main features, scope, and limitations.
- Integrate relevant RRL naturally into the discussion.
- Follow IEEE in-text citation formatting.
- Preserve the original technical scope of PigMENT.
- Avoid introducing features that are not part of the proposed system.Do not rewrite the entire Chapter 1. Only provide the completed Section 1.4."

Validation evidence:

- Compared the generated section with the original PigMENT concept and draft to check whetherthe intended objectives and system features were retained.
- Checked the discussion against the three main system features: pig growth monitoring andweight estimation, community livestock marketplace, and pig health information and geographic disease awareness.
- Reviewed the scope and limitations to ensure that automated weighing, disease diagnosis, and outbreak prediction were not presented as implemented capabilities.
- Checked the IEEE in-text citations against the supplied RRL reference list to identify incorrect

ormismatched reference numbers.
- Reviewed the final paragraphs for grammar, clarity, academic tone, and consistency with theproposed system.

AI Assistance Disclosure:
ChatGPT was used to assist in refining the academic writing, organizing the discussion, and integrating relevant literature into the proposed solution and scope of the study. The generated content was reviewed and checked against the project's original concept and reference materials.

08 Dashboard and My Pigs Interface Review

Exact prompt

Act as a professional UX/UI designer and front-end developer. Review the current PigMENT website, focusing on the Dashboard and My Pigs / Pig Growth Monitoring page.

PigMENT will be used by small-scale pig farmers in Marinduque. Some users are middle-aged or older and may not be very familiar with websites or digital tools. The current interface feels too plain and inactive. Improve it so it feels more welcoming, practical, and easy to use without making it cluttered.

UI/UX requirements
- Use larger, readable text and clear contrast.
- Make buttons large enough to recognize and click easily.
- Use icons together with text labels; do not rely on icons alone.
- Use simple, familiar wording.
- Make the most important actions visually prominent.
- Reduce unnecessary cards, decorative elements, and small controls.
- Add helpful instructions and clear empty states.
- Use consistent spacing, colors, buttons, and form styles.
- Make the layout responsive for desktop, tablet, and mobile.
- Avoid excessive animations, gradients, or complex visual effects.● Do not redesign the website as a generic corporate dashboard.

Dashboard improvements
Make the dashboard feel more useful and active:

- Add a friendly welcome message.
- Make “Register a Pig,” “Manage Pig Pens,” and “Create Marketplace Listing” prominent quickactions.
- Show only a few important summary items.
- Include a simple recent activity section.
- Use clear labels and short explanations.

Pig monitoring improvements
Improve the My Pigs page so farmers can easily understand their records:

- Make the search and filters easy to find.
- Use clear pig status labels.
- Make each pig record easy to scan.
- Include a visible “Register a Pig” button and a separate “Create Pig Pen” button.● Provide a helpful message when there are no registered pigs or pens.

Before editing, briefly identify the main usability problems in the current interface. Then update the existing

files while preserving the current project structure and any working functionality. Do not
08 Dashboard and My Pigs Interface Review (continued) introduce a
framework or backend.

09 Disease Awareness Map Prototype

Exact prompt

I want to develop the PigMENT Disease Awareness Map module first.

PigMENT is a web-based swine growth monitoring, marketplace, and disease awareness system for small-scale pig farmers in Marinduque.

Technology requirements:

- HTML5
- CSS3
- Vanilla JavaScript
- Leaflet.js
- OpenStreetMap

For this initial version, create a functional frontend prototype without PHP or MySQL.

FARMER INTERFACE:

1. Create an interactive map centered on Marinduque, Philippines.
2. Use Leaflet.js with OpenStreetMap tiles.
3. Display sample disease incident markers using clearly labeled fictional demonstration data.
4. Allow users to click a marker to view the disease type, municipality or barangay, incident date,and publication status.
5. Add filters for municipality and disease type.
6. Add a section explaining ASF symptoms, prevention, and biosecurity practices.
7. Include a button or page for submitting a suspected disease report.

ADMIN INTERFACE:

1. Create a separate admin dashboard within the existing PigMENT project.
2. Display submitted demonstration reports.
3. Allow the admin to inspect report details.
4. Include actions to mark reports as under review, reject them, or approve them for publication.
5. Only allow reports with an officially confirmed status and authorized confirmation details to bepublished as confirmed incidents.
6. When a demonstration incident is approved for publication, update the farmer-facing map sothat its marker becomes visible.

FUNCTIONAL REQUIREMENTS:

- Use JavaScript demonstration data for now.
- Keep the farmer and admin interfaces connected through the same temporary data source.
- Make the map markers and filters interactive.
- Make the layout responsive and consistent with the existing PigMENT design.
- Do not modify unrelated modules or redesign the existing dashboard.● Clearly label all demonstration records as fictional.

- Do not imply that the system automatically detects or diagnoses ASF.
- Keep the code organized so the temporary data can later be replaced with PHP API requestsand MySQL records.

Before making changes, inspect the existing project structure and identify where the new files should go. Explain your planned changes first, then implement the module without breaking existing features.

10 Login, Sign Up, Session Management, and Logout

Exact prompt

Act as a senior full-stack web developer working on our existing college project, PigMENT: A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque.

TASK:
Implement a fully functional Login, Sign Up, Session Management, and Logout experience by connecting the existing PigMENT frontend to its already-developed PHP authentication APIs.

CRITICAL REQUIREMENT:

DO NOT modify, recreate, reset, or restructure the existing MySQL database or PHP API files.

Our group is already working on database integration, and another developer is implementing the Marketplace. Your changes must not interfere with their work.

PHASE 1: INSPECT THE EXISTING PROJECT

Before implementing anything, inspect the current repository structure and carefully review:

- api/config.php
- api/csrf.php
- api/register.php
- api/login.php- api/logout.php
- api/session.php
- database/db.php
- src/index.html
- src/pages/my-pigs.html
- src/pages/marketplace.html
- src/pages/admin-disease-reports.html
- src/js/script.js
- src/css/style.css

Understand the existing API endpoints, expected request formats, session behavior, CSRF protection, account roles, and JSON responses.

Do not assume that a new authentication backend needs to be created.

PHASE 2: PROTECT THE EXISTING BACKEND

STRICTLY PROHIBITED:

1. Do not create, delete, rename, or modify MySQL tables or columns.
2. Do not generate or execute ALTER TABLE, DROP TABLE, or database migration commands.
3. Do not edit database/db.php.

4. Do not modify the existing authentication API endpoints.
5. Do not change existing API request or response structures.
6. Do not change PHP session configuration or CSRF verification.
7. Do not change user roles, permissions, or account-status behavior.
8. Do not modify Marketplace, pig management, weight records, pens, litters, ordisease-reporting PHP endpoints.
9. Do not replace the existing authentication system with Firebase, Supabase, JWT, or anotherauthentication service.
10. Do not introduce React, Vue, or another frontend framework.

If any existing backend behavior prevents correct frontend integration, report the exact issue and

the smallest proposed correction. Do not independently modify the backend.

PHASE 3: CREATE THE LOGIN AND SIGN UP FRONTEND

Create the necessary authentication pages inside the existing frontend structure.

Suggested files:

- src/pages/login.html
- src/pages/register.html
- src/js/auth.js

Additional frontend files may be created only when necessary.

Follow the existing PigMENT visual identity, typography, color palette, and styling.

DESIGN REQUIREMENTS:

- Professional, clean, and welcoming interface.
- Consistent with the existing PigMENT dashboard and Marketplace design.
- Suitable for small-scale farmers who may have limited technological experience.
- Clear labels and straightforward instructions.
- Responsive on desktop, tablet, and mobile devices.
- Avoid crowded forms, unnecessary decorations, and confusing controls.
- Use accessible HTML elements and visible keyboard focus indicators.
- Display validation and error messages near the relevant fields.
- Provide visible loading indicators during submissions.
- Prevent repeated submissions while a request is processing.
- Include password visibility toggles.
- Provide clear navigation between Login and Sign Up.

LOGIN PAGE:

Fields:

- Email address
- PasswordFunctions:
- Sign In button
- Show/Hide Password
- Link to Sign Up

- Field validation
- Clear login errors

SIGN UP PAGE:

Fields:

- First Name
- Last Name
- Email Address
- Password- Confirm Password
- Phone Number
- Municipality
- Barangay
- Address (Optional)

Functions:

- Create Account button
- Show/Hide Password
- Required-field validation
- Email validation
- Password confirmation
- Clear error messages
- Link back to Login

Match all submitted field names to the existing PHP API requirements.
Do not introduce registration fields that require new database columns.

PHASE 4: INTEGRATE THE EXISTING REGISTRATION API

Use the existing endpoint:

POST api/register.php

The endpoint expects JSON containing:

{

"first_name": "...",

"last_name": "...",

"email": "...",

"password": "...",

"phone_number": "...",
"municipality": "...",

"barangay": "...",

"address": "..."

}

IMPORTANT:

- The backend requires all fields except address.
- Password must contain at least 8 characters and must not exceed the backend's 72-byte limit.

- Confirm Password is for frontend validation only. Do not include it in the API payload.
- Follow the existing backend field-length restrictions.
- Do not send a role or account_status from the registration form.
- The existing backend automatically registers new users as active farmers.
- Do not allow users to select Admin during registration.
- Do not modify the existing password hashing implementation.
- Do not store passwords in browser storage.

REGISTRATION BEHAVIOR:

1. Validate the form.
2. Obtain a valid CSRF token from the existing API.
3. Submit the registration request using fetch().
4. Handle the existing JSON response format.
5. Display meaningful success and error messages.
6. Handle duplicate email, invalid fields, and connection errors.
7. After successful registration, direct the user to the Login page with a success message.

IMPORTANT:

The existing registration endpoint does not automatically create an authenticated login session. Do not assume users are logged in immediately after registration.

PHASE 5: INTEGRATE THE EXISTING LOGIN API

Use the existing endpoint:

POST api/login.php

Request body:

{

"email": "...",

"password": "..."

}

IMPORTANT:

- Use the existing PHP session-based authentication.
- Obtain the CSRF token before the login request.
- Send the CSRF token through the X-CSRF-Token request header.
- Include Content-Type: application/json.
- Use credentials: "same-origin" for requests to the same-origin PHP API.
- Do not store session IDs, authentication tokens, or passwords in localStorage or sessionStorage.- Do not implement fake frontend-only authentication.

The existing login API returns user information including:

- user_id
- first_name
- last_name
- email
- role

After successful login:

1. Recognize the authenticated session.
2. Retrieve the current session through api/session.php.
3. Refresh the CSRF token because login.php regenerates it after successful login.
4. Update the frontend to display the logged-in user's name.
5. Redirect farmers to the existing PigMENT dashboard.
6. Redirect administrators to the existing admin disease reports page, if appropriate for thecurrent project structure.
7. Handle inactive accounts, invalid credentials, and connection errors.

Do not create administrator accounts through public registration.

PHASE 6: SESSION MANAGEMENT

Use:

GET api/session.php

The session endpoint returns authenticated user information when the PHP session is valid. Implement a reusable frontend authentication helper responsible for:

- Checking login status.
- Retrieving authenticated user information.
- Handling session expiration.
- Showing the current user's name.
- Handling authenticated navigation.
- Handling account roles.
- Preparing authenticated requests with the correct CSRF token.

AUTHENTICATION REQUIREMENTS:

1. The dashboard must recognize an authenticated farmer.
2. My Pigs must recognize the current farmer's session.
3. Protected actions must not be presented as available to unauthenticated users.
4. If a session expires, display an appropriate message and redirect to Login when accessingprotected functions.
5. If an already authenticated user opens Login or Sign Up, redirect them to the appropriateexisting page.
6. Avoid redirect loops.
7. Preserve safe navigation paths when redirecting users to Login.
8. Never trust a role stored in browser storage.
9. Retrieve the actual role from the PHP session endpoint.
10. Keep authorization enforcement on the backend.

For the Marketplace:

- Preserve public listing browsing if supported by the existing API.
- Require authentication for farmer-only actions such as posting or managing listings.
- Do not modify Marketplace CRUD logic or API integration currently being developed by anothergroup member.
- If a necessary Marketplace integration change belongs to another developer, document therequirement instead of rewriting that module.

For the admin interface:

- Recognize the existing admin role.
- Do not show administrator-only navigation to ordinary farmers.
- Do not treat hiding an admin link or redirecting in JavaScript as sufficient security.
- Keep existing server-side authorization checks.
- Do not claim that a static HTML page is securely protected merely because it has a JavaScriptsession check.
- Do not rewrite the disease-reporting module.

PHASE 7: LOGOUT FUNCTIONALITY Use:

POST api/logout.php

Implement a visible Logout option for authenticated users.

LOGOUT BEHAVIOR:

1. Obtain or reuse a valid current CSRF token.
2. Submit the logout request.
3. Handle the API response.
4. End the authenticated frontend state only after successful logout or confirmed sessioninvalidation.
5. Redirect the user to Login.
6. Ensure protected features cannot continue being used without a valid session.
7. Do not delete the user's actual database records.
8. Do not clear unrelated frontend application data.

The existing PHP backend already handles session destruction. Reuse it.

PHASE 8: CSRF TOKEN HANDLING

Use the existing endpoint:
GET api/csrf.php

The endpoint returns:

{

"status": "success",

"csrf_token": "..."

}

Implement proper CSRF token handling:

1. Obtain a token before registration, login, or logout.
2. Pass the token in the X-CSRF-Token header.
3. Maintain the same PHP session through cookie-based requests.
4. Refresh the token after successful login because the existing backend regenerates it.
5. Handle expired or invalid tokens appropriately.
6. Never disable or bypass the backend CSRF protection.
7. Do not hardcode CSRF tokens.

PHASE 9: INTEGRATE WITH THE EXISTING UI

Update the existing dashboard and relevant pages so the authentication interface feels like part of PigMENT.

CURRENT FRONTEND LOCATIONS:

- src/index.html

- src/pages/my-pigs.html
- src/pages/marketplace.html
- src/pages/admin-disease-reports.html
- src/css/style.css

TASKS:

1. Replace placeholder account information with the authenticated user's name where applicable.
2. Add a clear Logout option.
3. Provide Login and Sign Up navigation when unauthenticated.
4. Ensure account information displays consistently on relevant pages.
5. Preserve the existing navigation structure.
6. Preserve all existing page functionality.
7. Do not redesign the dashboard or Marketplace.
8. Do not modify unrelated JavaScript business logic.

Keep all changes minimal and focused on authentication.

PHASE 10: VERIFY FILE PATHS AND LOCAL EXECUTION

The repository uses separate src/, api/, and database/ directories.
Inspect the actual folder structure and generate the correct relative paths.
Do not assume every HTML page is located at the repository root.

Ensure authentication requests work when running the project through a PHP-enabled local server such as XAMPP.

Do not use file:// URLs for testing PHP endpoints.

Do not hardcode absolute URLs tied to a particular developer's computer.

Ensure all paths continue working if the repository directory name differs between group members.

PHASE 11: TESTING AND VALIDATION

Verify these scenarios:

TEST 1:

Register a new farmer using valid information.

Expected: The registration API returns success and the new account is stored using the existing database structure.

TEST 2:

Register using an already registered email.

Expected: The existing API rejects the duplicate and the frontend displays an understandable error.

TEST 3:

Submit mismatched passwords.

Expected: The frontend rejects the submission before sending the API request.

TEST 4:

Login using valid farmer credentials.

Expected: Login succeeds and the farmer reaches the dashboard. **TEST 5:**

Login with an incorrect password.

Expected: Login is rejected and the user receives a clear error.

TEST 6:

Refresh the browser after successful login.

Expected: The PHP session remains recognized.

TEST 7:

Navigate between the dashboard and My Pigs.

Expected: The authenticated session remains valid.

TEST 8:

Logout successfully.

Expected: The session ends and protected actions are no longer available. **TEST 9:**

Access a protected action without authentication.

Expected: Login is required.

TEST 10:

Login with an existing admin account.

Expected: The application recognizes the admin role and does not grant farmer-only or administrator-only privileges incorrectly.

TEST 11:

Login with an inactive account.

Expected: The API rejects authentication.

TEST 12:

Submit a request with an invalid CSRF token.

Expected: The existing backend rejects the request. **TEST 13:**

Verify that all existing Marketplace, pig management, disease reporting, and navigation functionality remains unaffected.

TEST 14:

Check browser console errors, broken stylesheet links, incorrect redirects, and failed API requests.

Do not claim that a scenario passed unless it was actually executed and verified.

If live testing is unavailable, provide a manual testing procedure and label the scenario as unverified.

PHASE 12: FINAL REPORT

After implementation, provide:

1. List of frontend files created.
2. List of existing files modified.
3. Exact API endpoints reused.
4. Confirmation that no MySQL tables or attributes were changed.
5. Confirmation that existing PHP API files were not modified.
6. Explanation of the login, registration, session, and logout workflows.
7. Test results with Passed, Failed, or Not Tested status.

8. Remaining issues and limitations, if any.
9. Any compatibility concerns that should be communicated to the developer working onMarketplace.

IMPORTANT FINAL INSTRUCTION:

The goal is to implement the frontend authentication experience and connect it to the existing PHP backend WITHOUT changing the database schema, existing API contracts, or unrelated components.

Make only the necessary changes for Login, Sign Up, Session Management, and Logout.

Preserve the current working progress of PigMENT and the changes being implemented by other group members.

11 Admin Disease Zone Publication

Exact prompt

Inspect the existing PigMENT Admin Disease Zones publication workflow.

PROBLEM:

When I click Publish on an infected zone, I receive this message:

"An officially confirmed linked incident is required."

TASK:

1. Identify the exact PHP or JavaScript validation that produces this message.

11 Admin Disease Zone Publication (continued)

2. Check how infected zones are linked to disease reports.
3. Identify which report status and confirmation fields are required.
4. Verify whether the Admin Disease Reports workflow saves those required fields correctly.
5. Check whether the Edit Zone interface allows selecting a confirmed disease report.
6. Identify why the selected zone is failing the publication requirements.
7. If there is a genuine integration bug, fix it without changing the intended confirmation rules.

IMPORTANT:

- Do not bypass veterinary confirmation requirements.
- Do not automatically confirm reports.
- Do not modify the database schema.
- Do not reset existing reports or zones.
- Preserve existing PHP APIs wherever possible.
- Do not allow unverified ASF incidents to be publicly presented as confirmed.

Explain the exact cause and the correct workflow for publishing a verified infected zone.

12 Profile Page

Exact prompt

Complete the missing Profile page in our existing PigMENT system.

GOAL:

Create a simple, secure, user-friendly My Profile page where authenticated farmers can view and update their account information.

IMPORTANT:

We have limited Copilot usage. Make only the necessary changes and avoid unnecessary refactoring.

INSPECT:

- Existing sidebar and shared UI.
- Current authentication frontend.
- api/session.php
- api/register.php
- api/config.php
- api/csrf.php
- database/db.php

Use the actual project structure.

The current users table already includes:

- user_id
- first_name
- last_name
- email
- password
- phone_number
- address
- municipality
- barangay
- role

- account_status

Verify these existing fields before implementation.

PROFILE PAGE REQUIREMENTS:

1. My Information

Display:

- Full Name
- Email Address
- Contact Number
- Municipality
- Barangay
- Address
- Account Role

Use a simple profile layout with clearly labeled information.

2. Edit Profile

Allow users to update:

- First Name
- Last Name
- Phone Number
- Municipality
- Barangay
- Address

Keep email and account role read-only for this initial implementation.

Use an Edit Profile button that opens a simple editable form.

Include:

- Save Changes
- Cancel

3. Backend Integration

The current session.php endpoint returns only basic identity information. If no profile endpoint exists, create one minimal endpoint:

api/profile.php

Implement:

- GET: Retrieve the authenticated user's full permitted profile.- PUT: Update allowed profile fields.

Requirements:

- Reuse existing PHP session authentication.
- Require a valid session.
- Use the logged-in user_id from the server session.
- Use PDO prepared statements.
- Validate all input values.
- Require CSRF protection for updates.- Update only the authenticated user's account.
- Do not allow role or account_status modification.

- Do not expose the password hash.
- Do not accept arbitrary user IDs to access someone else's profile.

4. UI Behavior

- Load the actual account information.
- Allow editing supported fields.
- Save changes only after successful API confirmation.
- Show clear success and error messages.
- Refresh displayed account information after saving.
- Preserve the current session.
- Prevent duplicate submissions.
- Warn before leaving with unsaved changes.

5. Navigation

- Enable the existing Profile sidebar link.
- Keep the page consistent with the existing PigMENT UI.

- Include an appropriate Logout action using the existing logout functionality.
- Avoid duplicating the Login and Registration pages.

6. Design
   - Simple profile card.
   - Readable labels.
   - Clear Edit Profile action.
   - Responsive layout.
   - Accessible controls.
   - Consistent green and teal styling.
   - Avoid unnecessary statistics or decorations.

STRICT RESTRICTIONS:

- Do not modify existing MySQL tables.- Do not modify existing authentication APIs.
- Do not change login or registration behavior.
- Do not modify Marketplace or My Pigs functionality.
- Do not introduce a new authentication system.
- Do not create unnecessary PHP endpoints.
- Do not implement profile photo uploads or account deletion.

If the actual current schema does not support a requested field, report the limitation rather than modifying the database.

AFTER IMPLEMENTATION:

1. List files created.
2. List files modified.
3. Explain the API endpoint used or added.
4. Confirm no database schema changes.
5. Test displaying, editing, and refreshing profile information.
6. Test unauthorized access and expired sessions.
7. Report untested functionality honestly.

Keep the implementation small, secure, and focused on Profile.

13 Reports Page

Exact prompt

Complete the missing Reports page for our existing PigMENT web system.

GOAL:

Create a simple "My Farm Reports" page where authenticated farmers can view a summary of their actual livestock and marketplace records.

IMPORTANT:

Keep this task small and focused. We have limited Copilot usage.

INSPECT ONLY:

- Existing shared page layout and navigation.
- Relevant current CSS.

- api/pigs.php
- api/pens.php
- api/pen-assignments.php
- api/weight-records.php
- api/litters.php
- api/marketplace.php
- Existing frontend authentication helper.

Use the actual current paths and API response formats.

REQUIRED FEATURES:

1. Farm Summary
   - Total registered pigs.
   - Active pigs.
   - Sold pigs.- Deceased pigs.
2. Pig Pen
Summary - Total
active pens.
   - Assigned pigs per pen.
   - Current capacity and occupancy.
   - Do not calculate occupancy using unsupported assumptions about litter piglets.3. Growth Summary
   - Display latest available estimated weight for each pig.
   - Include measurement date.
   - Clearly label weights as estimates.
   - Do not invent missing measurement records or calculate a new formula here.
4. Litter Summary
   - Total recorded litters.- Recorded piglet counts.
   - Use existing litter records.
5. Marketplace Summary
   - Active listings.
   - Sold listings.
   - Inactive listings.
   - Use only the authenticated farmer's own listings.
6. Print Summary

   - Include a simple "Print Report" button.
   - Use a printer-friendly CSS layout and window.print(). - Do not install a PDF-generation library.

UI DESIGN:

- Use simple summary cards and clearly labeled sections.
- Keep the page consistent with PigMENT.
- Use readable text and spacing.
- Avoid complicated charts and crowded tables.
- Show clear empty states.
- Make it responsive and easy for farmers to understand.

NAVIGATION:

- Add a working Reports page to the sidebar.
- Preserve links to other modules.
- Do not duplicate the Pig Health "My Reports" disease-submission tracker.
- If useful, provide a link to that existing feature.

BACKEND:

- Use existing GET endpoints.
- Do not create a new database table.
- Do not modify existing APIs.
- Do not use hardcoded numbers or fictional records.
- Use authenticated, owner-scoped data only.
- Handle API errors and expired sessions correctly.
- If a statistic cannot be calculated from existing supported data, show it as unavailable andexplain why.

DO NOT:

- Modify My Pigs functionality.
- Modify Marketplace functionality.
- Modify Pig Health functionality.
- Rebuild the dashboard.
- Introduce frameworks.
- Change database tables or attributes.

AFTER IMPLEMENTATION:

- List files created and modified.
- Confirm that existing APIs were reused.- Confirm that the database was not modified.
- Report any unavailable statistics.
- Test navigation, real data loading, and printing.- Do not claim tests passed unless executed.

Implement only the Reports page and necessary navigation changes.

14 Pig Health and Geographic Disease Awareness

Exact prompt

Act as a senior full-stack PHP developer and UX/UI designer working on our existing project:

PigMENT: A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque.

TASK:

Finalize the Pig Health Information and Geographic Disease Awareness feature into a reliable, accessible, and properly integrated module.

This is one of PigMENT's THREE MAIN FEATURES.
The goal is to allow farmers to:

1. Access reliable pig health and disease prevention information.
2. View published geographic disease information on an interactive map.
3. Submit suspected disease reports.

4. Track the progress of their submitted reports.

Administrators must be able to:

1. Review submitted reports.
2. Record administrative decisions.
3. Record authorized veterinary confirmation details.
4. Publish eligible disease information.
5. Create and manage geographic monitoring and infected zones.

CRITICAL PROJECT CONTEXT:

PigMENT already contains a working frontend prototype for:

- Pig Health.
- Leaflet disease map.
- Farmer disease report submission.
- Admin Disease Reports.
- Geographic polygon drawing and management.

Do not recreate features that already exist.

The team is actively integrating PHP and MySQL.

Inspect the CURRENT repository first because other developers may have made newer changes since the previously reviewed ZIP.

PHASE 1 — INSPECT EXISTING IMPLEMENTATION

Locate and review:

- src/index.html
- src/js/disease-data.js
- src/js/disease-map.js
- src/js/admin-disease-reports.js
- src/pages/admin-disease-reports.html
- src/css/style.css
- api/config.php
- api/session.php
- api/csrf.php
- database/db.php

Also inspect any newer Pig Health pages, API endpoints, SQL definitions, or migration files.

Identify:

- Features already working.
- Incomplete features.
- Existing API endpoints.
- Database tables and columns.
- Existing authentication and authorization.
- Frontend data still using localStorage.
- Incorrect report or zone status handling.- Existing UI problems.

Before modifying anything, summarize the current implementation and identify which changes can be completed without database changes.

BACKEND AND DATABASE PROTECTION:

- Do not recreate or reset the database.
- Do not rename or delete existing tables.
- Do not modify unrelated PHP APIs.
- Do not change authentication or CSRF contracts.
- Do not interfere with Marketplace or My Pigs development.
- Do not replace Leaflet or OpenStreetMap.
- Do not introduce a frontend framework.
- Do not implement automatic disease diagnosis or outbreak prediction.

If the existing schema supports the required features, implement the missing disease-specific backend operations using the existing PHP conventions.

If tables or attributes are missing, propose the smallest additive schema changes for team approval. Do not automatically execute migrations.

Never pretend a backend-dependent feature is complete using localStorage or simulated API responses.

PHASE 2 — SIMPLIFY THE PIG HEALTH INTERFACE

Redesign only the Pig Health module to make it easier to navigate.

Organize it into four main sections:

1. Health Guide
2. Disease Awareness Map
3. Report an Illness
4. My Reports

Use clearly labeled tabs, navigation cards, or equivalent simple navigation.

Avoid displaying long educational text, reporting forms, and map controls all at once.

Use progressive disclosure:

Show essential information first, then allow users to open more detailed sections.

Keep the existing PigMENT color palette and visual identity.

Use:

- Large readable headings.
- Simple instructions.
- Clearly labeled buttons.- Meaningful icons.
- Spacious layouts.
- Responsive cards.
- Accessible forms.
- Helpful empty states.

Avoid:

- Crowded layouts.
- Excessive text.
- Unnecessary decorative elements.

- Confusing technical language.
- Unrelated dashboard statistics.

PHASE 3 — IMPROVE THE HEALTH GUIDE

The existing system contains basic ASF awareness information.

Improve the educational section while keeping the approved project scope.

Include accessible information covering:

A. AFRICAN SWINE FEVER

- General disease overview.
- Possible warning signs.
- Basic prevention measures.
- Biosecurity precautions.
- Recommended action when illness is suspected.

B. GENERAL PIG HEALTH

- Common pig health warning signs.
- Importance of monitoring changes in behavior, appetite, and physical condition.
- Practical preventive care and biosecurity principles.- When veterinary assessment is necessary.

C. BIOSECURITY GUIDE

- Farm cleanliness and disinfection.
- Visitor and equipment management.
- Isolation and movement precautions.- Reporting suspicious illness.

D. WHAT TO DO IF A PIG APPEARS SICK

- Contact an appropriate veterinary authority.
- Follow official veterinary guidance.
- Avoid presenting the application as a diagnosis or treatment service.

IMPORTANT:

Use accurate, properly attributed information from authoritative veterinary and agricultural sources, such as government veterinary agencies and established agricultural extension materials.

Do not invent medical claims or treatment recommendations.

Do not imply that all symptoms indicate ASF.

Do not include specific veterinary contact information unless it has been verified.

Clearly distinguish educational guidance from professional veterinary assessment.

Use short, readable explanations and expandable content instead of long paragraphs.
Include source references and review dates where practical.

PHASE 4 — FINALIZE THE DISEASE AWARENESS MAP

Preserve the existing Leaflet and OpenStreetMap implementation.

The current prototype already displays:

- Report markers.
- GeoJSON polygon zones.
- Confirmed zones.
- Monitoring zones.

- Resolved zones.
- Municipality filters.- Disease filters.
- Zone-status filters.

Improve the existing implementation instead of replacing it.

MAP REQUIREMENTS:

1. Display accurate geographic information.
2. Show only records authorized for public viewing.
3. Hide pending, rejected, and unpublished disease reports from the public incident map.
4. Clearly distinguish confirmed disease incidents from monitoring advisories.
5. Display resolved zones only when supported by a valid history and authorized resolution.
6. Provide readable map popups.
7. Preserve the current legend.
8. Ensure filters work consistently.
9. Display clear messages when no results match the filters.
10. Prevent private or unpublished report data from leaking through filter options.

Do not display fictional demonstration incidents together with real reports as if they are equally verified.

During development, label demonstration data clearly and keep it isolated. **PUBLIC LOCATION PRIVACY:**

Do not automatically display a farmer's exact home or farm coordinates.

Use appropriate public location precision for confirmed incidents and official advisories.

Private incident coordinates, when collected, must be protected according to the application's access rules.

Do not invent coordinates for submitted incidents.

PHASE 5 — FIX REPORT LOCATION SELECTION

IMPORTANT EXISTING BUG:

The previous disease-map.js implementation assigned the same latitude and longitude to every newly submitted report: latitude: 13.43 longitude: 121.96

This must be corrected.

Allow farmers to select the GENERAL INCIDENT LOCATION through the interactive map.

REQUIREMENTS:

1. Select municipality.
2. Enter or select barangay.
3. Select a point on the map.
4. Display the selected map pin.
5. Allow the farmer to adjust the selected point.
6. Validate the coordinates.
7. Confirm that the location is within the intended Marinduque reporting area.
8. Save the actual selected coordinates.
9. Do not automatically assign a fixed coordinate.
10. Explain that precise private locations are not automatically public.

Do not require GPS permission.

Manual map selection should remain available.

If location information is incomplete, provide a clear validation message.

Do not submit an inaccurate coordinate merely to satisfy required fields.

PHASE 6 — COMPLETE FARMER DISEASE REPORTING

Create a simple, reliable suspected disease report form.
Required information should follow the actual existing API and database schema.

Recommended fields:

- Suspected disease category.
- Municipality.
- Barangay.
- Incident date.
- Description of observed symptoms or unusual behavior.- Selected incident location on the map.

Use the authenticated farmer's user ID from the backend.

Do not ask the farmer to manually enter their account ID.

Do not allow the farmer to select Confirmed as the report status. **INITIAL REPORT STATUS:**

Pending Review.

REPORT VALIDATION:

- Required fields must be completed.
- Incident dates must be valid.
- Description must not be empty.
- Coordinates must be valid.
- Avoid duplicate submissions.
- Prevent repeated requests while submission is processing.- Display meaningful validation errors.

After successful submission:

- Show a confirmation message.
- Display the saved report in My Reports.
- Explain that submission does not mean the disease has been officially confirmed.

Do not claim that a report was submitted unless the server has confirmed that it was saved.

PHASE 7 — ADD MY REPORTS

THIS IS A REQUIRED NEW FEATURE.

Create a My Reports section where logged-in farmers can check the status of their own submissions.

Show:

- Report ID.
- Suspected disease category.
- Municipality and barangay.
- Incident date.
- Submission date.
- Current review status.
- Appropriate administrative feedback.

Use simple status labels:

- Pending Review.
- Under Review.
- Confirmed.- Rejected.

Keep publication status separate from review and confirmation status.

Allow users to open a report and view its submitted information. Do
not allow farmers to:

- Confirm their own reports.
- Approve publication.
- Edit veterinary confirmation details.
- View other farmers' private report details.

Preserve the original report and review history.

If report correction or withdrawal is supported, implement it without destroying the existing review history and only according to approved workflow rules.

Use actual database records, not a separate sample-data array.

PHASE 8 — COMPLETE ADMIN DISEASE REPORTS

Preserve the current admin report dashboard.
Improve it to provide a clear and reliable review workflow.

ADMIN QUEUE:

Display reports with:

- Report ID.
- Reported disease.
- Location.
- Incident date.
- Submission date.
- Current review status.- Inspect action.

Provide simple filtering by status and municipality.

REPORT DETAILS:

When an administrator selects a report, display the full submission in an organized detail panel.

Allow:

- Mark Under Review.
- Record review comments.
- Reject with a reason.
- Record external authorized veterinary confirmation.- Approve eligible information for publication.

DISTINGUISH TWO PROCESSES:

1. Administrative review.
2. Authorized veterinary confirmation.

An administrator must not automatically be considered a veterinarian.

When recording veterinary confirmation, capture supported information such as:

- Confirming personnel or office.

- Professional designation.
- Confirmation date.
- Official reference or supporting details, when available.

Do not fabricate official confirmation.

If the system cannot independently verify the professional's authority, do not claim that it does. Only authorized system roles should record confirmation supplied through an established official process.

Preserve the original submission and create an appropriate record of review actions when the backend supports it.

PHASE 9 — ENFORCE SAFE PUBLICATION RULES

This is critical because PigMENT displays disease-related public information.

Implement consistent rules on the PHP backend, not only in JavaScript.

REQUIREMENTS:

- Pending reports must not appear as confirmed incidents.
- Under-review reports must not appear as confirmed incidents.
- Rejected reports must not appear as confirmed incidents.
- Confirmed publication must require an appropriate authorized confirmation record.
- Publication must be a separate controlled action.
- Confirmation details must not consist only of a user-controlled frontend status.
- Only authorized administrative accounts may perform moderation and publication actions.- Farmers must not directly publish infected zones.

Monitoring advisories must be clearly labeled as monitoring, not confirmed infection. Resolved zones must retain appropriate historical context.

Do not change a report's clinical confirmation status merely because its public visibility changes.

Do not mark a report Confirmed simply because an administrator clicks Approve.

When a report is rejected, preserve the reason and relevant review history.

PHASE 10 — FINALIZE INFECTED-ZONE MANAGEMENT

The current project already uses Leaflet-Geoman for polygon drawing.

Do not recreate the entire zone management interface.
Improve the existing features:

1. Draw a geographic polygon.
2. Save the zone.
3. Edit the zone boundary.
4. Edit zone information.
5. Publish an eligible zone.
6. Update zone classification.
7. Resolve a previously confirmed zone when officially supported.
8. Unpublish or archive a zone when appropriate.
9. Prevent accidental deletion of relevant history.
10. Associate zones with appropriate reports or official source information.

ZONE INFORMATION:

Use the existing supported fields where available:

- Zone ID.
- Zone Name.
- Municipality.
- Barangay or affected location.
- Disease type.
- Severity.
- Zone status.
- Publication status.
- Date reported.
- Description.
- Associated report ID.
- Confirmation information.
- GeoJSON polygon geometry.

GEOMETRY VALIDATION:

- Accept the supported GeoJSON Polygon format.
- Use longitude, latitude coordinate order.
- Ensure the polygon is properly closed.
- Require a valid minimum number of distinct vertices.
- Reject invalid coordinates.
- Detect unsupported or malformed geometry.
- Avoid invalid self-intersecting polygons.
- Check the intended geographic reporting area.
- Do not create a polygon automatically from a single report marker.

IMPORTANT EXISTING ISSUE:

Inspect the polygon editing and drawing logic carefully.

Ensure that the SAVED geometry reflects the latest visible polygon boundary after the user edits or drags its vertices.
Do not save stale hidden-form coordinates after editing.

Ensure drawing controls remain usable after canceling or editing an existing zone.

PUBLICATION:

A monitoring zone must be labeled as an advisory and must not imply confirmed infection.
A confirmed infected zone must have authorized confirmation.

A resolved zone must not be fabricated without a valid earlier zone and appropriate resolution information.

Preserve the zone's record and history where possible.

PHASE 11 — INTEGRATE WITH PHP AND MYSQL SAFELY

The previously uploaded project had no completed disease-specific PHP endpoints.

Inspect the current repository to determine whether these have since been implemented.

Use the existing project conventions:

- PHP.
- PDO prepared statements.
- Session-based authentication.

- CSRF protection.
- JSON responses.
- Role-based authorization.

Required backend capabilities include:

FARMER:

- Create disease report.
- Retrieve own reports.
- View own report details and statuses.

ADMIN:

- Retrieve submitted reports.
- Review and update report status.
- Record confirmation details according to permissions.
- Publish or unpublish eligible incidents.
- Create, edit, and manage geographic zones.

PUBLIC:

- Retrieve published and appropriately verified incidents.
- Retrieve published zone geometry.
- Retrieve approved information for the public map.

Use existing database tables and APIs when they already support these operations.

DO NOT:

- Recreate existing tables.
- Rename established columns.
- Reset database records.
- Change unrelated API contracts.
- Implement parallel and inconsistent disease data stores.

If the required disease tables do not exist, prepare an exact proposed schema or additive migration and identify its compatibility impact.

Do not execute structural database changes without team approval.

Never use localStorage as the authoritative source for submitted real disease reports or official publication status.

If backend support is unavailable, display a truthful unavailable state instead of falsely reporting successful submission or publication.

PHASE 12 — SECURITY AND PRIVACY

- Require authenticated sessions for farmer report submission and private report tracking.
- Restrict report review and zone management to authorized administrators.
- Enforce permissions in PHP.
- Validate all input server-side.
- Require CSRF protection for state-changing requests.
- Use prepared SQL statements.
- Escape or safely render user-submitted descriptions.
- Avoid exposing private farm coordinates.

- Prevent farmers from modifying other farmers' reports.
- Prevent unauthorized users from publishing or changing zones.
- Never expose database errors or sensitive information to public users.

A hidden admin navigation link is not sufficient security. Use
the existing authorization helpers.

PHASE 13 — USER EXPERIENCE

Design for small-scale pig farmers, including users unfamiliar with digital systems.

Use:

- Clear navigation.
- Large readable text.
- Simple forms.
- Helpful explanations.
- Accessible buttons.
- Responsive layouts.
- Clear status labels.- Confirmation dialogs.
- Loading and error states.
- Proper keyboard accessibility.

Recommended wording:

"Pig Health"

"Health Guide"

"Disease Awareness Map"

"Report an Illness"

"My Reports"

"View Report"

"Pending Review"

"Under Review"

"Confirmed"

"Rejected"

"View Details"

Avoid unnecessarily technical terminology.

Make the report submission process understandable without requiring the farmer to understand database or GIS terminology.

Do not overwhelm the user with all controls simultaneously.

PHASE 14 — TESTING

FARMER TESTS:

1. Open Pig Health.
2. Read the Health Guide.
3. Open the disease map.
4. Apply municipality filters.

5. Apply disease filters.
6. Select a marker and view details.
7. Submit a suspected disease report.
8. Select an actual map location.
9. Verify the saved coordinates match the selected point.
10. Refresh and confirm the submitted report remains saved.
11. Open My Reports.
12. Confirm only the current user's reports appear.
13. Check report status.
14. Verify pending reports do not appear as confirmed public incidents.

ADMIN TESTS:

15. Log in as an administrator.
16. Open the report review queue.
17. Inspect a farmer submission.
18. Mark a report Under Review.
19. Reject a report and record the reason.
20. Record authorized confirmation details.
21. Test publishing without required confirmation.
22. Confirm publication is rejected when requirements are missing.
23. Publish an appropriately confirmed incident.
24. Verify the public map displays only eligible information.

ZONE TESTS:

25. Draw a polygon.
26. Save it unpublished.
27. Confirm it remains hidden from the public map.
28. Edit the polygon boundaries.
29. Save and reload the geometry.
30. Verify the saved polygon matches the edited geometry.
31. Publish an eligible zone.
32. Confirm the correct zone status and color.
33. Test resolved-zone handling.
34. Test invalid polygons.
35. Test unpublished and rejected records.

SECURITY TESTS:

36. Attempt to access admin operations as a farmer.
37. Attempt unauthorized report modifications.
38. Test invalid CSRF tokens.
39. Test expired sessions.
40. Confirm public APIs do not reveal private submitted reports or exact private farm locations.

REGRESSION TESTS:

41. Verify Login and Sign Up remain functional.
42. Verify My Pigs is unaffected.

43. Verify Marketplace is unaffected.
44. Verify dashboard navigation works.
45. Test desktop and mobile interfaces.
46. Check browser console and PHP errors.

Do not mark tests as passed unless actually executed.

PHASE 15 — FINAL IMPLEMENTATION REPORT

After implementation, provide:

1. Completed Pig Health features.
2. Missing or blocked features.
3. Existing files modified.
4. New files created.
5. PHP endpoints reused or created.
6. Database tables used.
7. Any proposed database changes.
8. Explanation of report submission and review.
9. Explanation of authorized confirmation.
10. Explanation of zone publication.
11. Explanation of geographic location validation.
12. Security and privacy safeguards.
13. Actual test results.
14. Confirmation that unrelated modules remain unaffected.

FINAL EXPECTATION:

Pig Health should function as a complete disease information, reporting, and geographic awareness module.

Farmers should be able to:

- Learn about pig health.
- View published disease-related information.
- Submit suspected cases.- Track their own reports.

Administrators should be able to:

- Review reports.
- Manage confirmation and publication.
- Draw and manage geographic zones.
- Publish accurate information with appropriate authorization.

Do not implement:

- Automatic disease diagnosis.
- Outbreak prediction.
- Disease spread simulation.
- Unverified official announcements.
- Invented veterinary confirmation.- Unnecessary additional systems.

Preserve the existing database and APIs wherever possible.

Do not claim the Pig Health feature is complete until the actual workflows have been tested and verified.

15 Community Livestock Marketplace Finalization

Exact prompt

Act as a senior full-stack PHP developer and UX/UI designer working on our existing project:

PigMENT: A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque.

TASK:

Finalize the PigMENT Community Livestock Marketplace into a complete, reliable, accessible, database-backed marketplace where farmers can post livestock for sale, manage their own posts, communicate with interested buyers, and mark listings as sold or unavailable.

IMPORTANT PROJECT CONTEXT:

Our team is already implementing the PHP and MySQL backend.

Marketplace development may contain newer changes than the previously reviewed source.

Inspect the CURRENT repository before changing anything.

Do not overwrite recent work from other group members.

Keep the existing Marketplace design direction but improve usability and incomplete functionality.

Do not rebuild the entire application.

PHASE 1 — INSPECT THE CURRENT MARKETPLACE

Locate and review the existing files, including:

- src/pages/marketplace.html
- src/js/script.js
- src/css/style.css
- api/marketplace.php
- api/listing-images.php
- api/config.php
- api/csrf.php
- api/session.php
- database/db.php

Also inspect any newer Marketplace-specific JavaScript, CSS, image upload handlers, and database migration files.

Identify the latest implementation before making changes.

The previously reviewed Marketplace backend already supported:

GET api/marketplace.php

GET api/marketplace.php?id={listing_id}

GET api/marketplace.php?mine=1

POST api/marketplace.php

PUT api/marketplace.php?id={listing_id}

DELETE api/marketplace.php?id={listing_id}

The existing backend used these listing statuses:

- active
- sold
- inactive

Verify these contracts still apply.
Before implementation, provide a short report identifying:

1. Existing working features.
2. Features still using sample JavaScript data.
3. Incomplete frontend-to-backend integration.
4. Existing API limitations.
5. Features requiring additional backend support.

DATABASE AND TEAMWORK RESTRICTIONS:

- Preserve existing MySQL tables and stored records.
- Do not recreate, truncate, or reset the database.
- Do not rename existing attributes.
- Do not alter existing authentication behavior.
- Do not rewrite working PHP endpoints unnecessarily.
- Do not modify My Pigs, Disease Awareness, or other unrelated modules.
- Do not change the current repository structure.
- Do not introduce an unrelated framework.
- Preserve existing API contracts wherever possible.

If a requirement genuinely needs a new field, endpoint, or table, explain the minimum necessary change and its compatibility impact before applying it.

Do not silently execute database migrations.

PHASE 2 — SIMPLIFY MARKETPLACE NAVIGATION

Organize the Marketplace into three main areas:

1. Browse Listings
2. My Listings
3. Post a Listing

BROWSE LISTINGS:
Available for viewing existing active livestock listings.

MY LISTINGS:

Available to authenticated farmers for managing posts they own.

POST A LISTING:

Available to authenticated farmers for creating a new livestock listing.

Use simple tabs, buttons, or page sections consistent with PigMENT's current navigation.

Do not overcrowd the Marketplace with unnecessary dashboard statistics.

Make the primary actions obvious.

PHASE 3 — COMPLETE BROWSE LISTINGS

Display actual published listings retrieved from the PHP API.
Do not use fictional sample listings as though they are actual farmer posts.

Each listing card should display:

- Main livestock photo, when available.
- Listing title.
- Pig type.
- Breed.
- Age.
- Estimated weight in kilograms.
- Asking price in Philippine pesos.- Municipality or general location.
- Seller name.- Availability status.
- View Listing button.

Use a clean responsive card layout.

Price and livestock information must be easy to read.

Do not display every detail on the listing card.

Use View Listing to reveal complete information.

SEARCH AND FILTERING:

Preserve and complete:

- Search by title, pig type, breed, and location.
- Filter by pig type.
- Filter by breed.
- Filter by weight range.
- Filter by municipality.- Reset filters.

Ensure search and filters work with actual records.

The previously reviewed public API returned a maximum of 50 listings.

Do not falsely present client-side filtering of the first 50 records as a complete search of all marketplace records.

If more records exist, propose or implement backward-compatible pagination and server-side filtering where authorized.

Provide:

- Loading state.
- Empty state.
- No matching results state.
- API error state.- Retry action.

PHASE 4 — COMPLETE VIEW LISTING

When a buyer selects View Listing, display a clean listing detail view.

SHOW:

1. Listing photos.
2. Listing title.

3. Asking price.
4. Pig type.
5. Breed.
6. Sex.
7. Age.
8. Estimated weight.
9. Description.
10. Municipality and barangay, when appropriate.
11. Seller's name.
12. Listing availability.
13. Contact Seller action.
14. Back to Listings action.

Clearly label estimated weights.

Do not display a seller's precise home address.

Do not expose private account information unnecessarily.

Use clear spacing and readable typography.

PHASE 5 — ADD CONTACT SELLER FUNCTIONALITY THIS IS A

REQUIRED FEATURE.

When a buyer views a listing, provide a prominent:

"Contact Seller"

action.

PURPOSE:

Allow interested buyers to discuss the livestock, price, availability, and transaction arrangements directly with the seller.

PREFERRED IMPLEMENTATION:

Use a simple, reliable direct-contact approach instead of building a full messaging system.

Supported options may include:

- Call Seller.
- Send SMS.
- Email Seller, if explicitly supported and approved.

Use actual seller contact information provided through an authorized backend response.

Do not hardcode phone numbers or email addresses.

IMPORTANT PRIVACY REQUIREMENTS:

The registration database may already contain a user's phone number, but registration does not automatically mean the farmer consented to publishing that number.

Do not expose users.phone_number or users.email through the public Marketplace endpoint without explicit seller consent.

Inspect whether the current project already has a safe way for sellers to choose which contact information is displayed.

If supported, allow the seller to enable their preferred contact method while publishing a listing.

If no consent mechanism exists, identify the minimum compatible backend/database change necessary.
Do not fake this feature or claim Contact Seller works when no authorized contact information is available.

If contact information has not been made available by the seller, display:

"Seller contact details are not available."

Do not introduce internal chat, payment processing, or conversation storage unless those features are separately approved.
Do not create fake message-sent success notifications.

CONTACT ACTION BEHAVIOR:

For phone numbers:

- Use a validated tel: link for calling.
- Use a validated sms: link when SMS is supported.
- Handle unsupported or unavailable contact methods gracefully.

For email:

- Use a validated mailto: link only when email disclosure is authorized.

Ensure the Contact Seller action is prominent and easy to understand.

PHASE 6 — COMPLETE POST A LISTING

Allow authenticated farmers to publish real livestock listings.

Use the existing POST api/marketplace.php endpoint.

The current backend expects listing fields such as:

- listing_title
- pig_type
- breed
- sex
- age_months
- estimated_weight
- asking_price
- description- municipality
- barangay
- pig_id (optional)

Map the existing frontend form correctly to these fields.

IMPORTANT DATA MAPPING:

- Sex values must match backend values: male or female.
- Age must be submitted as a numeric month value.
- Do not send an age string such as "5 months" if the API expects a number.
- Municipality and barangay must use their corresponding fields.
- Asking price must be a positive PHP amount.
- Estimated weight must be in kilograms.
- Optional values must be handled according to the actual API contract.

Remove unsupported form choices such as "Not Sure" if the backend does not permit them, unless an

approved compatible backend change exists. **LISTING FORM:**

Keep the form beginner-friendly.
Group related inputs into:

1. Pig Information.
2. Price and Location.
3. Photos and Contact Preferences.
4. Review and Publish.

If suitable, allow the seller to select one of their own registered pigs.

When a pig is selected:

- Retrieve only records owned by that farmer.
- Prefill supported pig details.
- Allow listing-specific fields where appropriate.
- Ensure breed and sex remain consistent with the linked pig.
- Do not permit inactive, sold, or deceased pigs to be newly listed if the backend prohibits this.

Also allow listing a pig that has not been registered in My Pigs, because the existing backend supports an optional pig_id.

LISTING PREVIEW:

Before publishing, show a preview containing:

- Main photo.
- Title.
- Pig details.
- Price.
- Location.
- Description.Provide:
- Back to Editing.- Publish Listing.

After successful API confirmation:

- Display a clear success message.
- Show the newly created listing in My Listings.
- Make it visible in Browse Listings if its status is active.
- Do not create duplicate posts when users click repeatedly.

Remove outdated messages claiming listings are session-only once actual API publishing works.

PHASE 7 — ADD MY LISTINGS MANAGEMENT

THIS IS A MAJOR REQUIREMENT.
Create a dedicated My Listings section for authenticated farmers.

Use:

GET api/marketplace.php?mine=1

Display ONLY the listings owned by the currently authenticated farmer.

Do not identify ownership based on the name displayed in the browser.

Use the backend's authenticated user ID and ownership checks.

Organize My Listings using simple status tabs:

1. Active
2. Sold
3. Unavailable**ACTIVE:**

Posts currently available for buyers.

SOLD:

Posts where the seller has confirmed that the deal was completed.

UNAVAILABLE:

Posts temporarily removed from public availability.

Each owned listing should display:

- Photo.
- Title.
- Asking price.
- Listing status.
- Date posted.
- View Details.
- Edit Post.
- Change Availability.

Keep the management interface simple and compact.

Do not display Edit or Remove controls on listings belonging to other sellers.

Backend ownership validation must remain mandatory.

PHASE 8 — EDIT YOUR OWN POST

Allow farmers to edit their existing posts.
Create a clear "Edit Post" action inside My Listings.

When selected:

1. Retrieve the correct listing.
2. Open the listing form with existing details filled in.
3. Allow modification of the supported attributes.
4. Validate all changes.
5. Show an option to Save Changes or Cancel.
6. Submit updates using the existing PUT API.
7. Refresh the listing after successful saving.

IMPORTANT:

The previously reviewed api/marketplace.php PUT operation requires the full listing payload, not just modified fields.
Inspect and follow the actual current API contract.

Do not accidentally reset listing_status to active during unrelated edits.

Do not generate a new listing ID when editing.

Do not change the seller ID.

Do not permit editing posts belonging to another farmer.

Preserve existing images unless the farmer explicitly changes them.

Clearly handle expired sessions, invalid values, and server errors.

PHASE 9 — MARK A LISTING AS SOLD THIS IS
A REQUIRED FEATURE.

Allow a seller to mark an active listing as SOLD after confirming that a deal was completed.

Provide an action:

"Mark as Sold"

When selected, show a confirmation dialog:

"Has the sale been completed?"

Explanation:

"Marking this listing as sold will remove it from available listings. It will remain in your listing history."
Actions:

- Cancel
- Confirm Sale

Only proceed when the seller confirms.

BACKEND BEHAVIOR:

Update listing_status to "sold".

Use the existing Marketplace update API and submit the complete required payload when necessary.

Ensure the backend confirms the operation before updating the UI.

After successful confirmation:

1. Remove the listing from active public browsing.
2. Move it to the Sold category in My Listings.
3. Display a Sold status.
4. Preserve the listing information for history.
5. Prevent buyers from attempting to contact the seller as though the pig were still available.
6. Do not remove the listing from the seller's records.

IMPORTANT:

Do not automatically change the linked pig's status in My Pigs unless this cross-module operation is explicitly designed, authorized, and safely implemented.

Report any mismatch between a sold marketplace listing and its linked pig record.

Do not implement payments, receipts, or financial transactions.

The seller's confirmation records a completed external deal; the platform is not verifying payment. **PHASE 10 — MARK A LISTING AS UNAVAILABLE**

Allow sellers to withdraw a listing temporarily.

Provide:

"Mark as Unavailable"

When selected, explain:

"This listing will be hidden from buyers until you make it available again." After

confirmation:

1. Update listing_status to "inactive".
2. Remove the listing from active public browsing.
3. Move it to the Unavailable category in My Listings.
4. Preserve its information and photos.
5. Allow the seller to reactivate the listing if the existing API permits it and the listing is stilleligible.

Provide:

"Make Available Again"

when appropriate.

Before reactivation, revalidate linked pig eligibility and any relevant backend restrictions.

Do not automatically reactivate listings marked as Sold.

Treat Sold and Unavailable as different statuses.

PHASE 11 — REMOVE YOUR OWN POST

Provide an option for removing a listing.

Prefer "Remove from Marketplace" as a safe deactivation action for ordinary users.

Explain the distinction:

Remove from Marketplace:

- Changes the listing to inactive.
- Hides it from buyers.
- Preserves the listing for later management.

Permanent Delete:

- Permanently removes the listing when supported.
- Requires explicit confirmation.
- May be restricted by related images or records.

IMPORTANT:

The previously reviewed DELETE endpoint rejects listings with attached listing_images records because image cleanup is not implemented.

Do not bypass that restriction.

Do not remove database references while leaving orphaned image files.

Do not silently delete active listing photos or other related records.

If safe permanent deletion is not supported, provide deactivation and report the missing deletion capability.

Never permit users to delete another farmer's posts.

PHASE 12 — COMPLETE LISTING IMAGE HANDLING

Inspect the existing:

api/listing-images.php

and:

public/assets/uploads/listings/

The earlier uploaded version had an empty listing-images.php file.

Check whether another developer has already implemented it.

If image handling is still incomplete:

- Identify the smallest secure implementation compatible with the current listing_images table.
- Preserve working Marketplace CRUD endpoints.- Do not invent unsupported database columns.
- Do not overwrite another developer's current work.

Required photo functionality:

1. Select a photo.
2. Preview before publishing.
3. Validate allowed image formats.
4. Validate file size.
5. Save uploaded images through a secure server-side upload process.
6. Store the resulting image path in the existing image table.
7. Display the saved image after refreshing.
8. Allow appropriate image management for the listing owner.
9. Prevent unauthorized image modification.
10. Safely handle cleanup when photos are removed.

Use randomly generated server filenames and appropriate file validation.

Never execute uploaded content as PHP.

Never trust client-provided filenames or paths.

If persistent upload is not yet available, clearly report the missing function rather than claiming that temporary object URLs are permanent photos.

PHASE 13 — MARKETPLACE ACCESSIBILITY AND UI DESIGN

The Marketplace is intended for small-scale pig farmers in Marinduque, including people who may have limited experience with technology.

Make the interface:

- Simple.
- Professional.
- Visually appealing.
- Responsive.
- Easy to navigate.
- Readable.
- Consistent with PigMENT's existing design.

USE:

- Clear headings.- Readable fonts.
- Meaningful icons.
- Accessible buttons.
- Large touch targets.
- Simple listing cards.
- Clear form sections.

- Consistent feedback messages.
- Visible loading states.
- Confirmation dialogs for important actions.- Helpful empty states.

Avoid:

- Crowded layouts.
- Excessive dashboard statistics.
- Large paragraphs in listing cards.
- Confusing technical terminology.
- Too many actions displayed simultaneously.
- Fake success messages.
- Unnecessary animations.
- Changing the entire website's styling.

IMPORTANT WORDING:

Use:

"Post a Listing"

"View Listing"

"Contact Seller"

"My Listings"

"Edit Post"

"Mark as Sold"

"Mark as Unavailable"

"Make Available Again"

"Remove from Marketplace"

Avoid technical or vague wording that may confuse farmers.

PHASE 14 — SECURITY AND RELIABILITY

Use the existing PHP authentication and session system.

Requirements:

1. Public users may browse active listings if supported by the current backend.
2. Posting requires an authenticated farmer.
3. Editing requires ownership of the listing.
4. Changing availability requires ownership.
5. Deleting requires ownership.
6. Never trust seller IDs sent by the frontend.
7. Use the existing CSRF token system for POST, PUT, and DELETE.
8. Use same-origin session cookies.
9. Handle expired sessions.
10. Handle backend errors honestly.
11. Never mark a post sold before successful server confirmation.12. Never expose private farmer information without explicit consent.
13. Render user-provided text safely to prevent XSS.
14. Prevent duplicate submissions.

15. Do not hardcode sample seller accounts.
16. Do not store authenticated user authority in localStorage.

Where a feature is unsupported by the existing API, identify its limitations rather than simulating success.

PHASE 15 — REQUIRED TESTING

Test these workflows:

BROWSING:

1. Load actual active listings.
2. Search by pig type or breed.
3. Filter by location.4. Filter by estimated weight.
5. Open listing details.
6. Verify inactive and sold listings are hidden from public active results.

POSTING:

7. Log in as a farmer.
8. Create a valid listing.
9. Publish it successfully.
10. Refresh the page.
11. Confirm the listing remains saved.
12. Confirm another user can see the active listing.
13. Test invalid form data.

OWNERSHIP:

14. View My Listings.
15. Confirm only the current farmer's posts appear.
16. Edit an owned listing.
17. Confirm changes persist.
18. Attempt to edit another farmer's listing.
19. Confirm the backend rejects unauthorized modifications.

SOLD WORKFLOW:

20. Mark an active listing as Sold.
21. Confirm the listing disappears from public active browsing.
22. Confirm it appears under Sold in My Listings.
23. Refresh and confirm the sold status remains saved.

UNAVAILABLE WORKFLOW:

24. Mark an active listing as Unavailable.
25. Confirm it disappears from public browsing.
26. Reactivate an eligible inactive listing.
27. Confirm it appears in active listings again.

CONTACT:

28. Open View Listing.
29. Test Contact Seller using an authorized contact method.
30. Confirm no private contact information is exposed without consent.
31. Confirm unavailable listings are not presented as actively for sale.

IMAGES:

32. Upload a supported image.
33. Confirm it remains after refreshing.
34. Test invalid image formats and file sizes.
35. Test unauthorized image modifications.

REGRESSION:

36. Verify Login and Sign Up still work.
37. Verify My Pigs remains functional.38. Verify the dashboard is unaffected.
39. Verify Disease Awareness is unaffected.
40. Test mobile and desktop layouts.
41. Check for browser console errors.
42. Check for PHP API errors.

Do not report tests as passed unless they were actually executed.

PHASE 16 — FINAL REPORT

After implementing the Marketplace improvements, provide:

1. Completed features.
2. Remaining missing features.
3. Files created.
4. Files modified.
5. Existing API endpoints reused.6. Any new API support required.
7. Any proposed database changes requiring approval.
8. Explanation of Contact Seller functionality and privacy.
9. Explanation of Sold and Unavailable status handling.
10. Explanation of permanent deletion restrictions.
11. Image upload status.
12. Test results.
13. Confirmation that unrelated modules were preserved.

FINAL EXPECTATION:

The PigMENT Marketplace must support a complete and practical livestock listing experience.

Farmers should be able to:

- Post a pig for sale.
- Upload and manage listing photos.
- View and edit their own posts.
- Remove a post from public browsing.
- Mark a completed deal as Sold.
- Temporarily make a listing unavailable.
- View their active, sold, and unavailable listings.

Buyers should be able to:

- Browse available pigs.
- Search and filter listings.
- View complete pig information.

- See seller information appropriate for public display.
- Contact the seller through a real, authorized communication method.

Do not introduce online payment processing, a shopping cart, or complex order management.

Preserve the current database and backend as much as possible.
Prioritize accurate functionality, seller privacy, simple interaction, and compatibility with the existing development work.

Do not declare the Marketplace complete until the required workflows are supported by actual backend operations and verified through testing.

16 Pig Weight Estimation and Quick Calculator

Exact prompt

TASK: Implement PigMENT's official pig weight-estimation formula and add a standalone Quick Weight & Selling Price Calculator.

PROJECT:

PigMENT — A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque.

IMPORTANT CONTEXT:

Pig Weight Estimation is one of the MAIN FEATURES of PigMENT. The
current project already contains:

- Pig registration and management.
- My Pigs and Pig Pen Management.
- PHP authentication and sessions.
- api/pigs.php.
- api/weight-records.php.
- An existing MySQL database.

The weight-estimation functionality must be restored and completed, not treated as an optional feature.

We also want a separate calculator that farmers can use without registering a pig.

CRITICAL RESTRICTIONS:

1. Do not modify existing MySQL tables or columns.
2. Do not recreate or reset the database.3. Do not modify existing PHP API contracts.
4. Do not replace working backend code.
5. Do not modify Marketplace or Disease Awareness.
6. Do not redesign the entire My Pigs page.
7. Preserve the simplified Pig Pen cards and other existing functionality.
8. Do not introduce an additional weight-estimation formula.
9. Do not introduce React, Vue, or another framework.

PHASE 1 — INSPECT EXISTING IMPLEMENTATION

Inspect the latest project structure and locate:

- The Welcome Page.- The My Pigs page.
- The Add a Pig form.
- The Growth Monitoring functionality.
- The current JavaScript files.
- api/pigs.php.
- api/weight-records.php.- Existing shared CSS.

Identify the existing weight-related fields, API request formats, and data structures before modifying anything.

PHASE 2 — IMPLEMENT THE APPROVED WEIGHT FORMULA

Use the following documented weight-estimation formula:

Weight (lbs) = (Heart Girth × Heart Girth × Body Length) / 400

Where:

- Heart Girth is measured in inches.
- Body Length is measured in inches.
- The initial result is in pounds.

ADJUSTMENT RULE:

If the INITIAL calculated weight is less than 150 pounds, add 7 pounds.

If the initial weight is 150 pounds or more, do not add the adjustment.

Apply the adjustment BEFORE converting to kilograms.

UNIT CONVERSIONS:

1 inch = 2.54 centimeters

1 pound = 0.45359237 kilograms

Ensure all conversions are accurate.
Use the original formula with measurements converted to inches, even when the user enters measurements in centimeters.

Do not use the number 400 directly with centimeter inputs.

Preserve full numerical precision during calculation and round only the displayed results to two decimal places.

FORMULA REFERENCES:

Utah State University Extension:

https://extension.usu.edu/davis/files/4H_Livestock_2012-04pr.pdf

University of Missouri Extension:

https://extension.missouri.edu/media/wysiwyg/Extensiondata/Pro/4h/Docs/Projects/PlantsAnimals/Swine/swine-nutrition.pdf

The formula is an estimation method, not an exact measurement. Do not claim that it has been independently validated for all Philippine native pig breeds.

PHASE 3 — CREATE A REUSABLE CALCULATION MODULE

Create one shared JavaScript calculation module.

Suggested responsibilities:

- Convert measurements to inches.
- Calculate estimated weight in pounds.
- Apply the under-150-pound adjustment.
- Convert estimated weight to kilograms.
- Calculate potential selling value.- Validate numerical inputs.

Use the SAME calculation functions for:

1. The standalone calculator.
2. Pig registration.
3. Recording new pig measurements.
4. Recalculating estimates when measurements are edited.

Do not duplicate the formula in multiple JavaScript files.

Keep the calculation module separate from database and API logic.

PHASE 4 — BUILD THE QUICK WEIGHT & PRICE CALCULATOR

Create a separate, accessible calculator within the existing PigMENT design.

Name:

Pig Weight & Price Calculator

Purpose:
Allow farmers to estimate a pig's weight and potential selling value WITHOUT registering or saving a pig.

The calculator must be available without requiring the user to create a pig record.

Prefer making it publicly accessible from the Welcome Page, including to visitors who have not signed in.

Do not expose authenticated farm records to public visitors.

FIELDS:

1. Heart Girth.
2. Body Length.
3. Measurement unit:

- Inches
- Centimeters

4. Selling Price per Kilogram (PHP/kg).

The price-per-kilogram field should be optional for weight estimation.

OUTPUT:

Display:

- Estimated Weight (kg).
- Estimated Weight (lbs).
- Selling Price per Kilogram.
- Potential Gross Selling Value (PHP).
- A short notice explaining that weights and selling values are estimates.

CALCULATOR DESIGN:

Use a simple, compact calculator layout.
Do not create a large complicated form.

The layout should

contain: A. Heading:

"Pig Weight & Price

Calculator" B. Short

description:

"Estimate your pig's weight and possible selling value without registering a pig." C.

Measurement inputs.

D. Estimated result panel.
E. Optional Reset button.
F. A small expandable "How to Measure" section.

The measurement guide should explain:

Heart Girth:

Measure around the pig's chest immediately behind its front legs.

Body Length:

Measure along the back from the midpoint between the ears to the base of the tail.

Use a simple, clear measurement illustration or appropriately licensed reference image if available.

Do not overwhelm the user with unnecessary technical terminology.

The calculator should use PigMENT's existing color palette and responsive design.

PHASE 5 — CALCULATOR BEHAVIOR

1. Allow measurements in inches or centimeters.
2. Ensure switching measurement units converts existing input values correctly instead ofchanging their physical meaning.
3. Calculate using the approved formula.
4. Apply the 7-pound adjustment only when required.
5. Display kilograms as the primary result.
6. Display pounds as secondary information.
7. Allow users to enter their intended price per kilogram.
8. Calculate estimated gross selling value.

FORMULA:

Estimated Selling Value (PHP) =

Estimated Weight (kg) × Price per Kilogram (PHP)
Do not confuse gross selling value with profit.

Do not use asking price as though it were price per kilogram.

Do not automatically save standalone calculator inputs.

Do not call pig registration or weight-record APIs from the public calculator.

The calculator should function entirely in the frontend and must not require database access.

Provide a Reset button that clears the inputs and results.

PHASE 6 — RESTORE AUTOMATIC WEIGHT ESTIMATION IN MY PIGS

Ensure the Add a Pig form includes a visible Weight & Measurements section. Include:

- Heart Girth (cm).
- Body Length (cm).
- Automatically calculated Estimated Weight (kg).
- Price per Kilogram (PHP/kg), when applicable.- Estimated Gross Selling Value.

The estimated weight must update when valid measurements are entered.

Display a clear message when measurements are incomplete.

Do not require the user to calculate the weight manually.

Do not remove existing pig registration fields.
Ensure all measurement inputs remain easy to access.

IMPORTANT:

When registering a pig, use the existing backend correctly.

api/pigs.php stores the pig's basic information. api/weight-records.php

stores measurements including:

- pig_id
- heart_girth
- body_length
- estimated_weight
- measurement_date- notes

The weight-records API expects measurements in centimeters and estimated weights in kilograms. If measurements are available during pig registration:

1. Register the pig through the existing pigs.php endpoint.
2. Obtain the saved pig ID.
3. Submit the measurement record using weight-records.php.
4. Store the calculated estimated weight in kilograms.
5. Associate the measurement with the correct pig.
6. Handle partial failures without creating duplicate pig registrations.

If measurements are intentionally omitted, allow registration according to existing API requirements and show that no weight estimate is available.

Do not change the database or backend API structure.

PHASE 7 — CONNECT WITH GROWTH HISTORY

When a registered farmer records new measurements:

1. Automatically calculate the estimated weight using the shared formula.
2. Save the new measurements and estimate through weight-records.php.
3. Associate the record with the correct pig.
4. Preserve all previous measurement records.
5. Update the pig profile's latest estimated weight.

6. Update the growth chart using real saved historical measurements.
7. Recalculate the estimate if an existing measurement record is edited.

Do not overwrite historical records when recording new measurements.

Do not fabricate missing historical data.

Do not save standalone guest calculator results to any registered pig.

PHASE 8 — CALCULATOR ACCESSIBILITY AND NAVIGATION

Make the calculator easy to discover.

WELCOME PAGE:

Add a visible action:

"Try Weight Calculator"

The action should open the standalone calculator without requiring login.

MY PIGS PAGE:

Add a clear action:
"Quick Weight Calculator"

The action should open the same calculator or reuse the same shared calculator component. Avoid creating separate calculation implementations.

If a farmer wants to save the measurement as part of a pig record, guide them to registration or the appropriate pig profile.

Never automatically register a pig from the calculator.

Preserve login, registration, and session routing.

PHASE 9 — VALIDATION AND ERROR HANDLING

Prevent:

- Empty required measurement inputs during calculation.
- Negative values.
- Zero measurements.
- Non-numeric or non-finite values.
- Invalid selling prices.
- Calculation results containing NaN or Infinity.- Duplicate measurement submissions.

Handle unrealistic measurements with understandable validation messages or warnings without inventing unsupported breed-specific limits.

Allow a weight estimate without a selling price.

If no selling price is provided, display the estimated weight but leave selling value unavailable.

If the estimated weight is invalid or unavailable, do not display a selling value.

Use Philippine peso formatting for currency.

Use clear, accessible error messages.

PHASE 10 — VERIFY FORMULA ACCURACY

Use these exact test cases:

TEST A:

Heart Girth = 40 inches

Body Length = 44 inches

Expected initial estimated weight:

176 pounds

Because this is not below 150 pounds, do not add 7 pounds.

Expected estimated weight:

Approximately 79.83 kilograms.

TEST B:

Heart Girth = 30

inches Body Length =

40 inches

Initial result:

90 pounds

Because 90 is below 150, add 7 pounds.

Expected estimated weight:
97 pounds, approximately 44.00 kilograms.

TEST C:

Enter equivalent measurements in centimeters.

Expected:

Same estimated result as the equivalent measurements in inches, subject only to display rounding. **TEST D:**

Change the price per kilogram.

Expected:

Weight remains unchanged; potential selling value updates correctly.

TEST E:

Clear the selling price.

Expected:

Weight is still calculated, but selling value is unavailable.

TEST F:

Open the standalone calculator while logged out.
Expected:

Calculator works without creating a user session or saving any pig records.

TEST G:

Register a pig with valid measurements.

Expected:

The pig and its associated initial weight record are saved through the existing APIs.

TEST H:

Record another measurement for the same pig.

Expected:

A new historical record is created and previous measurements remain available.

TEST I:

Refresh the registered pig's profile.

Expected:

Saved weight history remains available and uses the values stored in MySQL.

TEST J:

Check that My Pigs, Pig Pens, Login, Marketplace, and Disease Awareness remain unaffected.

Do not claim tests passed unless they were actually executed.

PHASE 11 — FINAL REPORT

After implementation, provide:

1. Files
created. 2.
Files
modified.
3. The exact formula implemented.
4. Formula references.
5. Explanation of unit conversions.
6. Explanation of the under-150-pound adjustment.
7. Description of standalone calculator behavior.
8. Description of My Pigs weight-estimation integration.
9. Explanation of how registered measurements are stored.
10. Test results.
11. Remaining limitations.
12. Confirmation that no existing MySQL tables or PHP API contracts were modified.

FINAL EXPECTATION:

PigMENT must provide two consistent ways to estimate pig weight:

A. Quick Weight & Price Calculator:
Available without registering a pig, with no required database storage. B.

My Pigs Growth Monitoring:

Automatic weight estimation, saved measurement records, historical growth tracking, and estimated selling value.

Both must use the exact same documented calculation method.

Do not remove or hide the automatic weight-estimation functionality again.

Preserve all existing farm-management and other project functionality.

17 My Pigs, Pens, Litters, and History

Exact prompt

Act as a senior full-stack developer and UX designer working on our existing system:

PigMENT: A Web-Based Swine Growth Monitoring, Marketplace, and Disease Awareness System for Small-Scale Pig Farmers in Marinduque. **TASK:**

Finalize the My Pigs feature, including pig registration and management, Pig Pen Management, litter records, weight monitoring, and historical records.

The goal is to make My Pigs a complete, reliable, easy-to-use farm management module, especially for farmers who are unfamiliar with technology.

I am not satisfied with the existing Pig Pen interface. It is too crowded and complex. Redesign it using simple expandable pen cards while preserving the system's existing functionality.

CRITICAL PROJECT CONTEXT:

The project already contains PHP APIs and an existing MySQL database. Other team members are actively working on Marketplace and other modules.

DO NOT rebuild the system, replace the backend, recreate tables, or modify unrelated functionality.

PHASE 1 — INSPECT THE EXISTING IMPLEMENTATION

Before making changes, inspect the actual repository and identify the relevant current files.

In the previously established structure, these include:

- src/pages/my-pigs.html
- src/js/script.js
- src/css/style.css
- api/pigs.php
- api/pens.php
- api/pen-assignments.php
- api/weight-records.php

- api/litters.php
- api/config.php
- api/csrf.php
- api/session.php
- database/db.php

The actual repository may have changed, so locate the current files rather than assuming these paths are unchanged.

Identify:

1. Working frontend functions.
2. Existing PHP API operations and request formats.
3. Existing database attributes referenced by those APIs.
4. Features still using sessionStorage or fictional sample data.
5. Frontend fields that do not match existing backend fields.
6. Incomplete features and broken workflows.
7. Features requiring database changes that have not been approved.

Create a short compatibility assessment before making modifications.

DATABASE AND API PROTECTION:

- Do not modify the existing database schema.
- Do not rename tables or attributes.
- Do not delete, recreate, or reset database records.
- Do not alter working PHP endpoint contracts.
- Do not change authentication, session, or CSRF behavior.
- Do not modify Marketplace or disease-reporting endpoints.
- Do not modify another developer's work.
- Do not invent API fields or assume unsupported database attributes exist.

Implement features using the existing PHP APIs whenever possible.

If a requirement cannot be correctly implemented with the existing APIs, identify the exact limitation and propose the smallest required backend or schema change in your final report. Do not silently implement those changes.

PHASE 2 — SIMPLIFY THE MY PIGS PAGE

Redesign the interface to emphasize only the information and actions farmers need most often.

RECOMMENDED STRUCTURE:

Page heading:

My Pigs

"Keep track of your pigs, pens, and growth records." Primary

action:

+ Register a Pig

Organize the page into clear sections or simple tabs:

1. My Pigs
2. Pig Pens
3. Growth & History

Keep Litter Management accessible through relevant sow profiles and pen details, rather than presenting it as another overwhelming dashboard section.
Avoid displaying too many statistics, forms, tables, and management buttons simultaneously.

Use progressive disclosure: show essential information first, then reveal additional details when requested.

Preserve existing application navigation and styling.

PHASE 3 — REDESIGN PIG PEN CARDS THIS IS

A MAJOR UI REQUIREMENT.

Replace the existing complex pen layout with simple, attractive, expandable cards.

COLLAPSED PEN CARD:

Each card should initially display ONLY:

- A recognizable pig icon or simple pig illustration.
- Pen name.
- Pen ID.

- Number of registered pigs currently inside.
- Occupancy versus maximum capacity, when available.- A clear expand/collapse indicator.

Example:

[Pig Icon]

Growing Pen

Pen ID: 002

5 / 8 spaces occupied
[View Pigs ▼]

Do not display long descriptions, extensive statistics, pig names, litter text, or management controls on every collapsed pen card.

Use a simple original or appropriately licensed pig silhouette icon. Avoid generic warehouse icons and childish cartoon styling.

Use compact, visually balanced cards with:

- Rounded corners.
- Clean spacing.
- Subtle shadows or borders.
- Clear typography.
- PigMENT's existing green and teal color palette.- Minimal decorative elements.

EXPANDED PEN CARD:

When a user selects a pen card:

1. Expand the selected pen.
2. Display the registered pigs currently assigned to that pen.
3. Show each pig as a small, organized information card or row.
4. Do not display all pig information as a long text paragraph.
5. Allow the user to collapse the pen again.

Each pig inside should show:

- Pig photo or pig icon.- Pig name.
- Pig ID.
- Breed.
- Sex.
- Latest estimated weight, if available.
- Current status.
- A clearly labeled "View Details" action.

Example:

Pen 002 — Growing Pen

Pigs Inside:

[Photo] Rosie

PG-024

Native Cross · Female

Latest estimate: 58 kg

[View Details]
[Photo] Bantay

PG-025

Landrace Cross · Male

Latest estimate: Unavailable

[View Details]

At the bottom of the expanded pen:

- Edit Pen
- Assign Pig
- View Assignment History

Keep these actions clearly organized and avoid crowding the card.

If a pen has no pigs, display:

"No pigs assigned to this pen yet."
Include a clear "Assign Pig" action.

ACCESSIBILITY:

- The expand/collapse control must work with mouse, touch, and keyboard.
- Use proper button elements and aria-expanded.
- Do not nest interactive buttons inside other buttons.
- Make only one pen expanded at a time on smaller screens.- Maintain readable information on mobile.

PHASE 4 — COMPLETE PIG PEN MANAGEMENT

Preserve and complete these operations:

1. Create Pen.
2. View Pen
Details.
3. Edit Pen.
4. Assign Pig.
5. Transfer Pig.
6. Remove Pig from Pen.
7. View Pen Assignment History.
8. Deactivate Pen.
9. Delete Pen only when allowed by existing backend rules.

PEN CREATION AND EDITING:

Use a short, beginner-friendly form containing:

- Pen Name.
- Pen Type.
- Capacity.
- Description (optional).
- Status, when appropriate.

Do not ask farmers to enter information that the system can calculate automatically.

For example:

- Current occupancy must be calculated from assignments.
- Empty is a derived condition, not a replacement for the backend's active/inactive status.
- Prevent invalid capacities.

PEN ASSIGNMENT:

- Only allow registered pigs belonging to the current farmer.
- Prevent duplicate active assignments.
- Prevent assigning pigs to inactive pens.
- Respect pen capacity.
- Allow transfer between existing active pens.
- Show clear success or error messages.
- Refresh the affected pen cards after saving.

PEN DELETION:

Respect existing API restrictions.

If a pen has assignment history and the API prohibits deletion, explain the reason and offer to deactivate the pen instead.

Never delete pig records just because a pen is removed.

IMPORTANT CAPACITY ISSUE:

The old frontend includes live litter piglets in its pen capacity calculation, but the current backend counts registered pig assignments.

Inspect this mismatch carefully.
Do not display one occupancy rule while enforcing another.

Do not silently modify the database or backend to resolve it.

Until the model is aligned, display only a capacity calculation that is accurately supported by the existing API, and explicitly report any limitation involving litter occupancy.

PHASE 5 — COMPLETE PIG REGISTRATION AND PROFILE MANAGEMENT

Ensure the following features are functional:

1. Register Pig.
2. View Pig Profile.
3. Edit Pig Information.
4. Update Pig Status.
5. Record Measurements.
6. View Growth History.
7. View Pen Assignment History.
8. Assign or Transfer Pen.
9. Access Related Litter Records.
10. Delete Pig only when safe.

REGISTRATION FIELDS:

Use the existing backend structure:

- Pig Code / Pig ID.

- Pig Name.
- Breed.
- Sex.
- Date of Birth OR Estimated Age in Months.
- Notes.

The backend currently supports male/female values, so do not submit an unsupported "Unknown" value.

Do not use a single text input to combine pig code and pig name.

Do not combine dates and age descriptions into one unstructured text field.

Use clear labels and simple explanations.

PHOTO FUNCTIONALITY:

Preserve pig photo selection and preview.

However, inspect whether the existing API actually supports persistent pig photo uploads.

If it does not, do not pretend the photo was saved permanently.
Clearly report that limitation instead of storing an image only in sessionStorage and claiming it is part of the saved pig record.

PIG PROFILE:

When a user selects a pig, display a clean profile using a simple detail panel or dedicated view.

Recommended structure:

Overview:

- Pig photo.
- Name and ID.
- Breed and sex.- Age.
- Current status.
- Current pen.
- Latest recorded estimated weight.

Growth History:

- Measurement records.
- Growth chart.
- Measurement dates.

Pen History:

- Previous pens.
- Assignment dates.
- Transfer and removal dates.

Litter Records:

- Associated litters, when applicable.

Actions:

- Edit Pig.
- Record Measurement.
- Transfer Pig.
- Update Status.

Keep the profile easy to navigate.

Do not show every detail in one long form.

PHASE 6 — IMPLEMENT EDITING AND SAFE RECORD MANAGEMENT

Allow farmers to edit pig information using a form populated with the current record.

REQUIREMENTS:

- Preserve the existing pig ID.
- Update the correct record.
- Do not generate a duplicate record during editing.
- Validate changes before submitting.
- Reflect successful changes immediately.
- Handle duplicate pig codes and other conflicts properly.
- Preserve historical measurement and pen records.- Warn before destructive operations.

Use the backend-supported statuses:

- Active.
- Sold.
- Deceased.

Display clear user-friendly labels, but send the backend's expected values.

Do not reuse the previous prototype statuses such as monitoring, paused, or needs-review as substitutes for the backend's actual pig_status. For deletion:

- Check dependencies.
- Explain if a pig cannot be deleted because other records depend on it.
- Prefer updating its status when that better preserves historical records.
- Never automatically delete connected measurements, litters, or marketplace records withoutexplicit supported behavior.

PHASE 7 — FINALIZE GROWTH MONITORING

The project's approved scope requires farmers to record heart girth and body length, maintain measurement history, display estimated weights, and estimate potential selling value.

Implement the following:

MEASUREMENT FORM:

- Select the registered pig.- Heart girth in centimeters.
- Body length in centimeters.
- Measurement date.
- Estimated weight in kilograms, when available.- Optional remarks.

Use the existing weight-records.php API.

IMPORTANT FORMULA REQUIREMENT:

The existing project does not contain a confirmed, validated weight-estimation formula.

Do not invent a formula or insert arbitrary constants.

If an approved, documented formula is already available elsewhere in the repository, verify its source and implementation before using it.

Otherwise:

- Record the measurements normally.
- Permit the existing API's optional estimated_weight field when manually supplied.
- Clearly label a manually supplied value.
- Show "Weight estimate unavailable" when appropriate.
- Report automated estimation as an outstanding requirement requiring an approved formula.

Do not claim the weight-estimation requirement is fully completed without a validated calculation.

GROWTH HISTORY:

- Keep every saved measurement as a dated record.
- Never overwrite old measurements when recording a new one.
- Show the latest measurement first in the history list.
- Allow users to inspect earlier measurements.
- Support editing an existing measurement through the available API.
- Warn before deleting a measurement.
- Keep the historical records associated with the correct pig.

GROWTH CHART:

Create a clear and readable chart based on saved measurements.

Use Chart.js if appropriate and consistent with the project's documented technology plan.

- Horizontal axis: Measurement date.
- Vertical axis: Estimated weight (kg), when available.
- Display real recorded values only.
- Do not invent missing data.
- Do not draw misleading trends when insufficient records exist.
- If only measurements are available, present those honestly instead of falsely labeling them asweights.

SELLING VALUE:

The project requires a potential selling value calculated using:

Estimated Weight (kg) × Farmer-entered Price per Kilogram (PHP)

- Clearly label the result as estimated gross selling value.
- Do not label it as profit.
- Do not calculate it without a valid estimated weight and valid price per kilogram.
- Display a helpful explanation when an estimate is unavailable.
- Determine whether the existing backend can store the price-per-kilogram preference.
- If it cannot, allow only appropriately labeled temporary calculation and report any requiredpersistence change.

Do not modify the database automatically.

PHASE 8 — COMPLETE LITTER MANAGEMENT

Preserve the existing litter-record feature.

Implement a simplified workflow accessible through the sow profile or an appropriate pen detail section.

REQUIREMENTS:

1. Create litter record.
2. View litter details.

3. Edit litter record.
4. Delete litter record when allowed.
5. Associate the litter with a registered female pig.
6. Display the mother sow's information.
7. Record farrowing date.
8. Record total born, born alive, and stillborn.
9. Validate total_born = born_alive + stillborn.
10. Display historical litter records.

Do not automatically register every piglet as an individual pig.

IMPORTANT:

The previous frontend associates litters with pens, but the current litter API does not expose the same pen assignment relationship.

Do not invent the relationship.

If pen-specific litter assignment cannot be persisted with the current database, retain correct litter management through the mother sow profile and document the missing integration.

Do not present a litter as belonging to a particular pen unless this association is supported by actual records.

PHASE 9 — ADD RELIABLE HISTORY VIEWS

History is a major requirement.
Provide clear historical information using the existing tables and APIs.

PIG GROWTH HISTORY:

Use saved weight_records.

Show:

- Measurement date.
- Heart girth.
- Body length.
- Estimated weight, if available.- Notes.

PIG PEN HISTORY:

Use saved pig_pen_history records through pen-assignments.php. Show:

- Pen name and ID.
- Date assigned.
- Date removed or transferred.
- Assignment status.
- Notes, when available.

PEN HISTORY:

When a farmer selects "View Assignment History" on a pen, filter the existing assignment records to show which pigs were assigned to that pen and when.

LITTER HISTORY:

Show saved litters associated with a mother sow.

Use a simple chronological layout.

Do not invent dates or historical events.
Do not claim a general edit audit trail exists unless the backend actually records changes to pig and pen information.

If such an audit trail is outside the existing database structure, identify it separately rather than fabricating history from updated_at.

PHASE 10 — CONNECT TO THE EXISTING BACKEND

Replace frontend-only farm data with real API-backed data wherever the existing backend supports it.

Use:

- api/pigs.php
- api/pens.php
- api/pen-assignments.php
- api/weight-records.php
- api/litters.php

Use the existing PHP authentication session and CSRF token handling.

REQUIREMENTS:

- Use the actual JSON API contracts.
- Use fetch() with the appropriate methods.
- Use cookies for the existing same-origin PHP session.
- Include CSRF headers on state-changing requests.
- Preserve current authentication behavior.
- Do not expose another user's records.
- Do not use localStorage or sessionStorage as the authoritative source for database-backed farmrecords.
- Do not mix fictional demonstration records with actual registered farmer data.
- Do not hardcode frontend records as though they were retrieved from MySQL.
- After successful changes, reload or update the appropriate affected data.
- Avoid unnecessary duplicate API calls.
- Handle 401, 403, 404, 409, 422, and server errors appropriately.
- Do not show a success message before the server confirms success.

If the backend is unavailable, show a clear error instead of silently falling back to demonstration records.

Use safe DOM rendering for user-provided text.

PHASE 11 — USER-FRIENDLY INTERACTION

The target users include small-scale farmers who may not be familiar with complex digital systems.
DESIGN REQUIREMENTS:

- Simple language.
- Large readable labels.- Clearly named buttons.
- Minimal forms.
- Helpful empty states.
- Obvious success and error messages.
- Consistent navigation.
- Responsive layouts.
- Touch-friendly controls.

- Accessible keyboard interaction.
- Strong color contrast.
- Proper field labels.
- Clear loading states.
- Confirmation before destructive actions.
- No unnecessary animations.
- No excessively long tables.
- No crowded dashboard-style interface.

Examples of user-friendly labels:
"Register a Pig"

"View Pig Details"

"Edit Pig"

"Record Weight"

"Transfer to Another Pen"

"View History"

"Edit Pen"

"Assign Pig"

Avoid technical labels such as:

"Execute CRUD"

"Update Entity"

"Modify Assignment Record"

When opening a form, allow the user to cancel safely.
Do not lose unsaved changes without warning when switching views.

Use consistent icons and spacing.

Keep the existing PigMENT visual identity.

PHASE 12 — PROTECT OTHER MODULES

This is a team project under active development.

DO NOT:

- Redesign the dashboard.
- Modify Marketplace data or its PHP API.- Modify disease reporting or the awareness map. - Reorganize repository folders.
- Replace authentication.
- Introduce an unrelated frontend framework.
- Change shared CSS in a way that breaks other pages.- Remove existing working modules.

The current script.js may contain code for more than one page.

If necessary, isolate My Pigs logic into a dedicated page-specific JavaScript file, but preserve other page behavior and existing script references.

Make the smallest safe changes necessary.

PHASE 13 — REQUIRED TESTING

Test the complete feature using the existing backend. **PIG RECORDS:**

1. Register a pig.
2. View the saved pig.
3. Refresh and confirm it remains available.
4. Edit the pig.
5. Verify changes were saved.
6. Attempt to register a duplicate code.
7. Update the pig status.
8. Test safe deletion or dependency handling.

PIG PENS:

9. Create a pen.
10. Confirm it appears as a compact card.
11. Expand the pen.
12. Confirm only its assigned pigs appear.
13. Collapse it.
14. Edit pen information.
15. Assign a pig.
16. Transfer a pig to another pen.
17. Confirm both pens display accurate assignments.
18. Attempt to exceed pen capacity.
19. View pen assignment history.
20. Test deletion and deactivation restrictions.

GROWTH MONITORING:

21. Record a measurement.
22. Record another measurement.
23. Confirm both appear in history.
24. Confirm the chart uses actual saved data.
25. Edit a measurement.
26. Check the estimated selling value when valid inputs are available.
27. Verify unavailable estimates are labeled honestly.

LITTER MANAGEMENT:

28. Register a litter for a female pig.
29. Validate incorrect piglet counts.
30. Edit the litter.
31. View historical litter records.

RELIABILITY:

32. Refresh after saving records.
33. Test with a second farmer account.
34. Confirm farmers cannot access another owner's records.
35. Test failed API requests.
36. Test invalid and expired sessions.

37. Check browser console errors.
38. Test desktop and mobile layouts.
39. Verify that Marketplace and other modules remain functional.Do not mark tests as passed unless they were actually executed.

PHASE 14 — FINAL REPORT

After implementation, provide:

1. A summary of all improvements.
2. The final My Pigs feature list.
3. The new Pig Pen UI structure.
4. Files created and modified.
5. Existing APIs reused.
6. Working features that were verified.
7. Features that remain blocked by backend/schema limitations.
8. Exact proposed API/database changes, if any, without implementing them.
9. A testing checklist with Passed, Failed, and Not Tested results.
10. Confirmation that existing database tables and PHP API contracts were not modified.
11. Confirmation that unrelated modules were preserved.

FINAL EXPECTATION:

My Pigs must feel like one complete and consistent farm-management feature.

The Pig Pen section should be visually simple:

- Compact pig-themed cards.
- Pen name and ID.
- Occupancy information.
- Click to reveal pigs inside.
- View and edit individual pig information.
- Assign and transfer pigs.
- Edit pens.
- View real historical records.

Prioritize correct functionality, reliable record handling, ease of use, and preservation of existing team progress.

Do not claim the module is fully complete while required functions remain unsupported by the actual backend.